claude-windows / 73a3938e-1da3-4e28-bfa7-346a15718301

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301.jsonl`
- SHA-256: `d21b95f75d56fba036ef140334a85cea90ee980b2646649ebe9a02f3fadb17d9`
- Source modified: `2026-06-30T04:05:30+00:00`
- Imported at: `2026-07-05T16:48:18+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `73a3938e-1da3-4e28-bfa7-346a15718301`

Transcript

1. user (2026-06-22T12:12:45.472Z)

In backend/worker/__main__.py, `cmd_train` cannot match labels to videos when labels use the new canonical GCS naming `labels/<YYYY-MM-DD>_<name>.rallies.csv` (see docs/DATA_IN_GCS.md §7.1 + §7b blocker #1).

Root cause (verified): cmd_train derives the per-clip key as `name = fname.replace(".rallies.csv","").replace(".csv","")` (backend/worker/__main__.py:384), e.g. `2026-06-22_GX020094`. It then looks up the video via `video_by_stem.get(name)` (:387), where `video_by_stem` is keyed by the bare video stem `os.path.splitext(vname)[0]` = `GX020094` (:378). The date prefix means the stems never match ? every clip hits the "no video for ... skipping" branch (:389) ? `worker train --trajectory-source detector --corpus-uri gcs://khelsutra-rally-corpus` aborts with "<2 videos". (synth-train + all eval are unaffected ? they don't join videos.)

Fix: strip a leading ISO date prefix when deriving the clip name, e.g. after line 384 add `name = re.sub(r"^\d{4}-\d{2}-\d{2}_", "", name)` (import re). Better: factor the label?clip-name derivation into a small helper and unit-test it (a dated label name and a bare label name both map to the bare clip stem, and join to `videos/<stem>.mp4`). Confirm `import re` and that the spec `name` used downstream is the clean (date-stripped) name.

Verify: run `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` + `ruff check .` + `mypy backend training` (the repo's pre-LGTM gate). Branch off origin/master (Khelsutra/badminton-highlight-indexer), one PR for this item only. The cloud corpus to test against: gs://khelsutra-rally-corpus (labels/<date>_<name>.rallies.csv + videos/<name>.mp4; only videos/mahadevpura_1.mp4 is uploaded so far).

2. assistant (2026-06-22T12:12:51.843Z)

I'll start by following the repo conventions: pull latest, read the target code, then branch off `origin/master` explicitly. Let me gather context in parallel.

3. user (2026-06-22T12:12:53.024Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
```

```
    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
    (validator registry ? segmenter ? report), never a forked pipeline.
    """
    from __future__ import annotations

    import argparse
    import csv
    import json
    import logging
    import os
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.config import (
        StartupCheckError,
        enforce_startup_checks,
        load_config,
        probe_cuda_available,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
```

```
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
        cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true no-op
        (returns True, ZERO work) when no deployment.profile is selected.

        The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily imports
        torch, and pre-M5 the worker's infer/train path was torch-free (detection is delegated to a
        WSL/native subprocess). Probing only when a profile is set keeps the default-OFF path a true
        no-op of work, not just of output."""
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF path
        try:
            enforce_startup_checks(config, cuda_available=cuda, logger=logger)
            return True
        except StartupCheckError:
            return False  # already logged at ERROR by enforce_startup_checks


    def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
        """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once the
        outputs are durably in the bucket (the backend bucket-polls the done-marker). The dispatched
        container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-dispatch.

        A remote job that never writes a completion marker (a failed/hung container) makes the bucket
        poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote dispatch
```

```
        timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
        from backend.infrastructure.execution_policy import PolicyTimeout

        spec = ComputeJobSpec(
            video_uri=args.video_uri, out_uri=args.out_uri,
            segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
        )
        try:
            result = compute.run_infer(spec)
        except PolicyTimeout as e:
            logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
            return 4
        logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
                    result.video_id, result.returncode, result.outputs_uri)
        return result.returncode


    def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
                           status: str, storage_cfg: dict, scratch: str) -> None:
        """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
        dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
        try:
            marker = {"video_id": video_id, "returncode": returncode,
                      "segment_count": segment_count, "status": status}
            local = os.path.join(scratch, "_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
```

```
config=storage_cfg)
179         print(f"[worker] pulled {args.video_uri} -> {local_video}")
180
181         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
182         video_id = compute_video_id(local_video)
183
184         # 3. VALIDATE (same registry as the main CLI).
185         val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
186         failures = [r for r in val_results if not r.passed]
187         db = Database(os.path.join(scratch, config.output.db_name))
188         db.add_video(video_id, local_video, "failed" if failures else "processed",
189                      [r.model_dump() for r in val_results])
190
191         segments: List[dict] = []
192         segmenter_actual = None
193         segmentation_ran = False
194         status = "failed"
195         try:
196             if failures:
197                 print("[worker] validation FAILED: "
198                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
199                 return 2
200             seg_name = args.segmenter or config.indexing.default_segmenter
201             segmenter_cls = segmenter_registry.get(seg_name)
202             if not segmenter_cls:
203                 print(f"[worker] unknown segmenter: {seg_name!r} "
204                       f"(have {list(segmenter_registry.list_available())})")
205                 return 2
206             segmenter_actual = seg_name
207             result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
208             segmentation_ran = True
209             if isinstance(result, Failure):
210                 print(f"[worker] segmenter FAILED: {result.message}")
211                 return 3
212             segments = result.value if hasattr(result, "value") else result
213             status = "success"
214             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
215             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
216             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
217             if not segments:
218                 detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
219                                  or getattr(config.indexing, "detector_impl", None) or
"auto")
220                 logger.warning(
221                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
222                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
223                     "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
224                     "the WASB env produced no detections, or the input resolution differs
from the tuned "
225                     "proxy resolution. Re-run with -v for the native command + frame
counts.",
226                     video_id, segmenter_actual, detector_impl,
227                 )
228         finally:
229             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
```

```
            emit_indexer_report(
                db=db, video_id=video_id, video_path=local_video, config=config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=False, segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        out_local = os.path.join(scratch, "outputs", video_id)
        os.makedirs(out_local, exist_ok=True)
        _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
        _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(out_local)):
            uri = f"{out_prefix}/outputs/{video_id}/{fn}"
            storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
            pushed.append(uri)

        print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
        # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
        # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
        if getattr(args, "done_uri", None):
            _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
        return 0


    #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
    #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
    _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")


    def _probe_video_fps_width(path: str):
        """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.

        The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
        fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
        cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
        inference report (it flags the fallback) but corrupting for REAL training data. So
here we
        probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
        rather than guesses.
        """
        import json
        import subprocess
        try:
```

```
            out = subprocess.check_output(
                [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
                 "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
                stderr=subprocess.DEVNULL).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(f"[worker] unknown segmenter: {args.segmenter!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                  f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
            return None
        try:
            runner, _ = seg._resolve_runner(config.get("indexing", {}))
        except NotImplementedError as e:
            # e.g. detector_impl='native' for a family without a native runner (tracknet, or
            # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
            print(f"[worker] detector not available: {e}")
            return None
        ok, msg = runner.healthcheck()
        if not ok:
            print(f"[worker] detector environment not ready: {msg}")
            return None
        print(f"[worker] detector ready ({args.segmenter}): {msg}")
        return runner


    def cmd_train(args: argparse.Namespace) -> int:
        """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
        to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.

        Two trajectory sources (the train pipeline + harness are identical either way):
          * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
```

```
synthetic
338             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
339           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
340             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
341             is the M3.5 "first real training" path; only the trajectory source flips.
342         """
343         import subprocess
344
345         config = load_config(args.config) if args.config else load_config()
346         _apply_overrides(config, args.set)
347         if not _run_startup_checks(config):
348             return 2
349         storage_cfg = config.storage.model_dump()
350
351         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
352         labels_dir = os.path.join(scratch, "labels")
353         traj_dir = os.path.join(scratch, "trajectories")
354         videos_dir = os.path.join(scratch, "videos")
355         os.makedirs(labels_dir, exist_ok=True)
356         os.makedirs(traj_dir, exist_ok=True)
357
358         corpus = args.corpus_uri.rstrip("/")
359         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
360                             if u.lower().endswith(".csv"))
361         if len(label_uris) < 2:
362             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
363                   f"under {corpus}/labels/")
364             return 2
365
366         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
367         runner = None
368         video_by_stem: dict = {}
369         if args.trajectory_source == "detector":
370             os.makedirs(videos_dir, exist_ok=True)
371             runner = _resolve_train_detector(args, config)
372             if runner is None:
373                 return 2
374             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
375                 vname = storage.parse_uri(vuri).name
376                 if not vname.lower().endswith(_VIDEO_EXTS):
377                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
378                 video_by_stem[os.path.splitext(vname)[0]] = vuri
379
380         print(f"[worker] trajectory source: {args.trajectory_source}")
381         specs: List[dict] = []
382         for uri in label_uris:
383             fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
384             name = fname.replace(".rallies.csv", "").replace(".csv", "")
385             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
386             if args.trajectory_source == "detector":
387                 vuri = video_by_stem.get(name)
388                 if not vuri:
389                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
390                     continue
391                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
```

```
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [sys.executable, "-m", "training.gen0.harness",
               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
        if args.floor:
            floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
                           if "://" in args.floor else args.floor)
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
        return 0
```

```
448
449
450     def build_parser() -> argparse.ArgumentParser:
451         p = argparse.ArgumentParser(prog="python -m backend.worker",
452                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
453         p.add_argument("-v", "--verbose", action="store_true",
454                        help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
455         sub = p.add_subparsers(dest="cmd", required=True)
456
457         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
458         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
459         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
460         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
461         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
462         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
463         pi.add_argument("--max-frames", type=int, default=None)
464         pi.add_argument("--done-uri", default=None,
465                         help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
466                              "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
467                              "Default-OFF: omit it for the normal local run.")
468         pi.add_argument("--set", action="append", default=[], metavar="k=v",
469                         help="config override, e.g. --set indexing.skip_ai_handoff=true")
470         pi.set_defaults(func=cmd_infer)
471
472         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
473         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
474                         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
475         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
476         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
477         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
478                         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
479                              "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
480                              "(needs videos/ in the corpus; honours detector_impl).")
481         pt.add_argument("--segmenter", default="wasb_hybrid",
482                         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
483         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
484         pt.add_argument("--config", default=None)
485         pt.add_argument("--scratch", default=None)
486         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
487         pt.add_argument("--frame-width", type=float, default=1280.0,
488                         help="synth trajectory frame width (detector probes the video)")
489         pt.add_argument("--set", action="append", default=[], metavar="k=v")
490         pt.set_defaults(func=cmd_train)
491         return p
492
493
494     def _setup_logging(verbose: bool) -> None:
```

```
495         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
496         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
497         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
498         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
499         entry), not at import, so importing the module never reconfigures a caller's
logging."""
500         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
501         # configured logging at import), which would leave the level at WARNING and re-
suppress the
502         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
503         logging.basicConfig(
504             level=logging.DEBUG if verbose else logging.INFO,
505             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
506             datefmt="%H:%M:%S",
507             force=True,
508         )
509
510
511     def main(argv=None) -> int:
512         args = build_parser().parse_args(argv)
513         _setup_logging(getattr(args, "verbose", False))
514         return args.func(args)
515
516
517     if __name__ == "__main__":
518         sys.exit(main())
519
```

4. assistant (2026-06-22T12:12:56.293Z)

Now let me pull latest, check the docs referenced, and look at existing worker tests.

5. user (2026-06-22T12:13:00.946Z)

```
---STATUS---
```

work/golden-corpus-hygiene...origin/work/golden-corpus-hygiene

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

6. user (2026-06-22T12:13:00.972Z)

```
1       # Data in GCS ? the corpus source-of-truth map
2
3       **Status:** LIVE / in-progress (migration not complete). **Owner-gated** decisions in
§7.
4       **Last inventoried:** 2026-06-22 against `gs://khelsutra-rally-corpus`.
5       **Goal (owner, 2026-06-22):** run **all** training + inference from the cloud with **all
data in GCS**,
6       so the local Windows box (and the intermittently-mounted external drive) is **no longer
a blocker**.
7
8       > **? Other sessions: this is where the data lives.** Pull from `gs://khelsutra-rally-
corpus`, not
9       > from anyone's `C:` / `F:` / Google Drive. If something you need isn't in the bucket
yet, it's in the
10      > **gap table (§5)** ? flag it / migrate it, don't hard-code a local path.
11
12      This doc is the **data map**. The *machine* runbooks are
```

[`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md)
13   (spin up a box) and [`deploy/gcp/README.md`](../deploy/gcp/README.md) (provision GPU +
train/infer).
14   The cloud engine itself was built by the (archived) DECOUPLED_COMPUTE project
15   (`docs/archives/past_projects/DECOUPLED_COMPUTE/`); this doc does **not** redesign it ?
it records
16   *where the bytes are* and *what's still missing*.
17
18   ---
19
20   ## 1. The bucket
21
22 | | |
23 |---|---|
24 | Bucket | **`gs://khelsutra-rally-corpus`** (single corpus bucket) |
25 | Project | `gen-lang-client-0306026826` (billing *Khelsutra Billing*) |
26 | Region | `asia-south1` (Mumbai), UBLA, co-located with the GPU VMs |
27 | Auth | SA `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`); key at `~/.gcp/rally-worker-sa.json` (local only, **never
committed**). Interactive: `gcloud auth login`. |
28 | Size today | ~2.06 GB (mostly past dry-run reels/outputs ? **not** the full corpus) |
29
30   The engine is **backend-agnostic by URI** (`backend/storage/ref.py`): `gcs://` (or
`gs://`),
31   `s3://`, `gdrive://`, or a bare local path. Pick the backend with **config + creds, no
code change**.
32
33   ---
34
35   ## 2. Canonical layout (the target contract)
36
37   The cloud worker's input/output contract (`backend/worker/__main__.py`; lineage:
38   `DECOUPLED_COMPUTE/04`):
39
40 | Prefix | Holds | Who reads/writes it |
41 |---|---|---|
42 | `videos/` | source videos / proxies (`<clip>.mp4`) | **input** to `worker infer` &
`worker train` |
43 | `labels/` | golden rally labels (`<clip>.rallies.csv`) | **input** ground truth to
`worker train` + all eval |
44 | `trajectories/` | cached WASB trajectory CSVs | ? **eval/litmus input** (GPU-free),
consumed via `manifests/golden.json`. **NOT** read by `worker train` (it synths or re-extracts)
|
45 | `weights/<version>/` | trained Gen-0 bundles (`weights.json` + a run `manifest.json`)
| **output** of `worker train` |
46 | `models/` | upstream pretrained weights (`wasb_badminton_best.pth.tar`) | **staged to
a local path** (`$WASB_WEIGHTS` / `~/models/?`) for the detector ? not read from `gs://` in
place |
47 | `outputs/<video_id>/` | per-video inference outputs (intervals, etc.) | **output** of
`worker infer` |
48 | `highlights/`, `promo/` | rendered reels + promo artifacts | output / demo |
49
50   > **Note on the manifest.** `worker train` does **not** read a pre-staged corpus
`manifest.json` ? it
51   > *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
**synthetically
52   > by default** (`--trajectory-source synth`, CPU-mock) or for real via opt-in
`--trajectory-source
53   > detector` (needs `videos/`). A `manifest.json` only appears as an **output** inside
each
54   > `weights/<version>/` bundle. The **eval/litmus** path is what wants a stable, path-
resolvable manifest
55   > (§7.3) ? and `manifests/golden.json` is currently consumed by **no code yet** (eval is
local-FS only;
56   > see Known blockers).

```
57
58      ---
59
60      ## 3. What's actually in the bucket today (2026-06-22)
61
62      > **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
`labels/<date>_<name>.rallies.csv`
63      > (= original-6 + `GX010137`/`GX010141` + 7 new; the 7 old `_proxy`/bare dupes were
deleted), and
64      > `manifests/golden.json`. The table below is the **pre-Phase-1** snapshot; the
remaining gap is **videos**
65      > (14 of 15) ? see §6/§7b/§8.
66
67      | Prefix | Present | Drift / notes |
68      |---|---|---|
69      | `labels/` | 7 files: `GX010128`, `mahadevpura-1`, + `Badminton BXH 2_proxy`, `Boxhill
Doubles_proxy`, `GX020094_proxy`, `GX030094_proxy`, `mahadevpura-2_proxy` | ? **inconsistent
naming** (`_proxy` on some, not others); ? **missing** the original-6 labels + `GX010137`,
`GX010141` |
70      | `videos/` | 3: `GX030094_proxy.mp4`, `mahadevpura-1.MP4`, `mahadevpura_smoke_180s.mp4`
| ? corpus videos/proxies almost entirely absent |
71      | `trajectories/` | ? | ? **does not exist** (every trajectory is local-only) |
72      | `models/` | `wasb_badminton_best.pth.tar` | ? (? WASB-C3 weight-provenance still an
open legal item before any external share) |
73      | `weights/` | `0.1.0-l4/` | ? first real L4 train bundle |
74      | `outputs/` | one `<video_id>/`, `parity/` | ? dry-run outputs |
75      | `highlights/`, `promo/` | reels + promo | ? |
76
77      **Drift summary:** the bucket has extra prefixes (`highlights/`, `promo/`, `models/`
alongside
78      `weights/`) and **inconsistent `labels/` naming**, and is **missing the bulk of the
corpus inputs**
79      (`videos/`, all `trajectories/`, most `labels/`). It reflects past *dry runs*, not a
complete corpus.
80
81      ---
82
83      ## 4. How the cloud consumes it
84
85      Provisioning + the verified commands live in
[`deploy/gcp/README.md`](../deploy/gcp/README.md) §3?§5.
86      The data-facing entry points (`backend/worker/__main__.py`):
87
88      ```bash
89      # config (no secrets in config; auth via GOOGLE_APPLICATION_CREDENTIALS):
90      #   { "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826"
} } }
91
92      # INFERENCE on one cloud video -> outputs/<video_id>/ in the bucket
93      python -m backend.worker infer \
94        --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
95        --out-uri   gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
96
97      # TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds
manifest on-box,
98      # runs training.gen0.harness, pushes weights/<version>/):
99      RALLY_DETECTOR_IMPL=native python -m backend.worker train \
100       --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
101       --version <v> --trajectory-source detector --segmenter wasb_hybrid --config
config.json
102     ```
103
104     A bare/relative path resolves under a configured `input_root`/`input_backend`
105     (`backend/storage/ref.py::resolve_input_uri`), so workflows can be written bucket-
relative.
```

---

5. The gap ? what's still local-only (why the box is still a blocker)

All of this is **tiny except the originals** ? the metadata that blocks cloud eval is < 25 MB total:

Data	Where it is now	Size	In bucket?	Needed for
20 WASB trajectory CSVs	`C:\?\Annotation Setup\Collect\Trajectories\`	**17.5 MB**	? none	eval/litmus (cached). *(No `cached` train source exists ? train synths or detector-extracts.)*
golden manifest	`output/human_lovo_manifest.json` (C:-absolute paths)	4.5 KB	?	eval corpus definition
original-6 labels	`F:\[Khelsutra]?\Training\Golden Labelled\*.rallies.csv`	~12 KB	?	train + eval
7 new-clip labels	repo `output/*.rallies.csv` (gitignored) **+** F: archive (backed up 2026-06-22)	~12 KB	? partial/inconsistent	train + eval
source videos / proxies	F: archive + Google Drive golden folder	**100s of GB**	? 3 only	infer + train (real WASB extraction)

> The corpus *metadata* (trajectories + labels + manifest, ~25 MB) is what actually blocks GPU-free
> cloud eval and the served Gate-A litmus ? and it's trivially small to migrate. The **videos** are the
> only large migration, and they already exist on **Google Drive** (the golden-corpus folder), so they
> can go **Drive ? bucket from a cloud box**, keeping the local machine out of the path entirely.

---

6. Migration plan (close the gap)

Phase 1 ? corpus metadata (?25 MB, do first, removes the eval blocker). Stage all trajectories,
all labels, and a **bucket-relative eval manifest** into `gs://khelsutra-rally-corpus`. Pending the §7
naming decision. Sketch:

```bash
# trajectories (decision-free; absent today)
gcloud storage cp "C:\?\Collect\Trajectories\*.csv" gs://khelsutra-rally-corpus/trajectories/
# labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
# eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object (§7.3)
```

Phase 2 ? videos ? `videos/<name>.mp4` (bucket name = the eval `name`, consistent with
`trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see `scratch/copy_videos_to_gcs.ps1`):

Source	Clips	Size
Drive golden folder `?/Golden Label Videos Request - June 15/[Done] Video N/` (**proxies**)	`mahadevpura_1/2`, `GX010128`, `GX020094`, `GX030094`, `Badminton_BXH_2`, `Boxhill_Doubles` (7)	~13.5 GB
F: archive (**full MP4s** ? proxy-first before upload)	`GX010137`, `GX010141`, + original-6 (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`, `testlarge_short`, `mahadevpura_singles`) (8)	~50 GB

Two ways to run it:
- **Decoupled (preferred ? no local box):** `rclone` on a cloud VM with a Google-Drive

remote ?
151   `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
152   (one-time owner step: `rclone config` ? `gdrive` remote OAuth). Or `gdown <file-id>`
on the box for
153   the shared folder. This is the path that actually removes the local box.
154   - **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
155   -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses
local bandwidth.
156
157   **Priority:** the 7 Drive **proxies** first (already 1080p, sufficient for detection).
The 8 large F:
158   full MP4s should be **proxied to 1080p first** (cheaper storage + faster cloud decode)
before upload.
159   ? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
160   are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-
dupe-then-delete
161   (same as the label cleanup). Track per-clip in §8.
162
163   **Phase 3 ? make eval/litmus read GCS (real code, NOT a config flip).** The eval drivers
164   (`served_gate_a.evaluate_video`, `calibrate_wasb.load_golden_gts`,
`TrackNetRunner.parse_trajectory_csv`)
165   open `spec["trajectory"]`/`spec["labels"]` with plain `open()` and gate on
`os.path.isfile()` ? they
166   have **no storage-layer fetch**, so a `gs://` manifest cannot be consumed today. Phase-3
must thread
167   `storage.fetch`/`resolve_input_uri` through those paths first. Then: re-probe
`fps`/`frame_width` from
168   the bucket videos (don't trust the copied values ? a wrong fps silently mis-aligns every
label), and
169   **re-measure the served Gate-A litmus on the GCS 15-clip set as a [[launch-report-
convention]] event**
170   (dual-set RAW+golden no-regression, not a silent eval-set swap ? see PR #236).
171
172   ---
173
174   ## 7. Decisions
175
176   1. **`labels/` naming ? LOCKED (owner, 2026-06-22):** `labels/<YYYY-MM-
DD>_<name>.rallies.csv`, where
177   `<name>` is the eval/manifest clip name and `<YYYY-MM-DD>` is the label's **authoring
date** =
178   the *earliest* mtime across all known copies (C: working copy + F: archive ? robust
to copy-resets;
179   a restored C: copy shows today, the F: archive keeps the real date). Fixes the
`_proxy` inconsistency
180   and disambiguates recycled GoPro names by date. *(Future, when videos are in the
bucket: optionally
181   add a `video_id`/MD5 sidecar for the strongest keying ? deferred, needs hashing each
video.)*
182   2. **Cache `trajectories/` ? YES, done 2026-06-22.** `trajectories/<name>.csv` cached so
cloud/CI eval +
183   the served Gate-A litmus run **GPU-free** (training can still regenerate via
`--trajectory-source detector`).
184   3. **Eval manifest in GCS ? done 2026-06-22.** `gs://khelsutra-rally-
corpus/manifests/golden.json`
185   (15 clips, `gs://` trajectory + label paths). The **local** litmus keeps its local-
path manifest;
186   flipping the litmus to read the GCS manifest is Phase 3.
187   4. **Originals migration scope ? OPEN** ? proxies-only vs full 4K, and how much of the
15-clip golden
188   set vs the wider archive. Source = Google Drive ? bucket from a cloud box (not the
local box).

```
189
190     > Phase-1 was run via a dry-run planner (resolve every clip's trajectory + label, stamp
the label's
191     > authoring date, build the `gs://` manifest) ? `gcloud storage cp` (planner kept in
gitignored
192     > `scratch/`; promote it to `scripts/` if corpus-sync recurs).
193     > ? **Cleanup DONE 2026-06-22:** the 7 pre-existing inconsistent label objects
(`*_proxy.rallies.csv`,
194     > bare `GX010128`/`mahadevpura-1`) were confirmed **byte-identical (MD5)** to the new
canonical names
195     > (content also on C:/F:) and **deleted** ? `labels/` now holds exactly the 15 canonical
objects.
196
197     > ? **gcloud gotcha (Phase-2):** `gcloud storage cp` treats `[...]` as a glob, so source
paths under
198     > `[Done] Video N/` (Drive) or `[Khelsutra] ?` (F:) fail with "matched no objects." The
owner-box
199     > script (`scratch/copy_videos_to_gcs.ps1`) stages such paths through a bracket-free
temp file; the
200     > `rclone` cloud path has no such issue.
201
202     ---
203
204     ## 7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)
205
206     1. **Dated labels break `worker train --trajectory-source detector` (code-verified).**
`cmd_train`
207        derives the join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`,
then looks up
208        `video_by_stem.get(name)` against bare video stems (`GX020094`) ? so every clip is
skipped and train
209        aborts with `<2 videos`
([backend/worker/__main__.py:384](../backend/worker/__main__.py):384 vs
210        :378/:387). **Fix (recommended):** strip a leading `^\d{4}-\d{2}-\d{2}_` from the
label-derived
211        `name` in `cmd_train` (a ~1-line change, own PR + test) ? preserves the locked human-
readable naming
212        AND unblocks detector-train. *(`synth` train + all eval are unaffected ? they don't
join videos.)*
213     2. **Phase-3 eval-on-GCS is net-new plumbing, not a flag** ? see §6 Phase-3. Eval is
local-FS only.
214     3. **`manifests/golden.json` is consumed by zero code today** (train ignores it; eval
can't fetch
215        `gs://`) ? it's correct-but-dormant until blocker #2 lands. Add a tiny manifest-
validator + a CI job
216        that authenticates to GCS and runs the litmus against it, or the DoD ("CI reads only
from `gs://`")
217        stays unverifiable.
218     4. **Gating infra (owner-only, not done):** GPU `GPUS_ALL_REGIONS` quota is still **0**
(no VM ever
219        created ? `deploy/gcp/README` §2); cloud-box secrets (SA JSON + rotated
`GEMINI_API_KEY` + WASB
220        weights staged from `models/`) aren't set up persistently. No real cloud train/infer
runs until these.
221     5. **Reproducibility:** the migration + copy helpers live in gitignored `scratch/`
222        (`migrate_golden_to_gcs.py`, `copy_videos_to_gcs.ps1`) ? promote to `scripts/` so
Phase-2/re-syncs
223        are repeatable from the repo, and add a referenced **proxy-generation** step for the
8 large F: originals.
224
225     ## 8. Progress tracker
226
227     | Item | Status |
228     |---|---|
229     | Bucket + SA + provisioning (deploy/gcp) | ? built |
```

```
230     | Cloud worker `infer`/`train` by URI | ? built (`backend/worker`) |
231     | This data-map doc | ? 2026-06-22 |
232     | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
233     | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
234     | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
235     | Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified
dupes, deleted) |
236     | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
**1/15** demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the
owner-box script |
237     | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
238     | Phase 3: eval/litmus default to the GCS manifest | ? |
239
240     **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
241     reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.
242
243     ---
244
245     ## Related
246     - [`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md) ? spin up a box (storage backends in
§7).
247     - [`deploy/gcp/README.md`](../deploy/gcp/README.md) ? GCP GPU provisioning + train/infer
(§5).
248     - [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) ·
[`PER_VIDEO_WORKSPACE.md`](PER_VIDEO_WORKSPACE.md) ? locality tiers + `outputs/<video_id>/`.
249     - `docs/archives/past_projects/DECOUPLED_COMPUTE/` ? the cloud-engine design lineage.
250
```

7. assistant (2026-06-22T12:13:04.220Z)

The doc confirms the bug precisely. Now let me find the existing worker tests to match conventions.

8. user (2026-06-22T12:13:05.868Z)

```
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
fa2342c feat(nightly): E2E engine reel-count regression on a GCP GPU VM (emailed) + cost (#315)
7a40434 feat(eval): golden fixtures P3 ? 6 curated, trimmed, de-attributed REAL Tier-0 fixtures
+ --max-frames trim (#317)
---
1b33f55 docs(data): correct DATA_IN_GCS per adversarial review (blockers + accuracy)
53467c9 docs(data): record stale-label cleanup + set up Phase-2 video copy
6ae3503 docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)
8884c9f docs(data): add DATA_IN_GCS.md ? the corpus source-of-truth map (GCS)
```

9. user (2026-06-22T12:13:06.298Z)

```
.claude\worktrees\bhi-execpolicy\tests\test_job_worker_loop.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_job_worker_loop.py
.claude\worktrees\tier1-windowing\tests\test_job_worker_loop.py
.claude\worktrees\w2-safeguard\tests\test_job_worker_loop.py
.claude\worktrees\hardchop\tests\test_job_worker_loop.py
.claude\worktrees\boxhill17\tests\test_job_worker_loop.py
.claude\worktrees\pr-stitch\tests\test_job_worker_loop.py
.claude\worktrees\gap-doc\tests\test_job_worker_loop.py
.claude\worktrees\resweep\tests\test_job_worker_loop.py
.claude\worktrees\recdoc\tests\test_job_worker_loop.py
tests\test_job_worker_module_seam.py
tests\test_worker.py
tests\test_job_worker_loop.py
```

10. user (2026-06-22T12:13:07.061Z)

```
Found 7 files
docs\DATA_IN_GCS.md
backend\worker\__main__.py
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\PER_VIDEO_WORKSPACE.md
docs\DESKTOP_APP_PLAN.md
tests\test_worker.py
tools\generate_highlights.py
```

11. assistant (2026-06-22T12:13:10.710Z)

`tests/test_worker.py` is the home for these tests. Let me read it, focusing on the train-related tests.

12. user (2026-06-22T12:13:11.120Z)

```
1       """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3       `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4       segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5       on the droplet against MinIO.
6       """
7       import json
8
9       import pytest
10
11      from backend.config import load_config
12      from backend.worker import __main__ as wmain
13
14
15      # ---------------------------------------------------------------- pure helpers
16
17      @pytest.mark.parametrize("raw,expected", [
18          ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19      ])
20      def test_coerce(raw, expected):
21          assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24      def test_apply_overrides_on_typed_config():
25          cfg = _fresh_config()
26          wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                       "sport=tennis"])
28          assert cfg.indexing.skip_ai_handoff is True
29          assert cfg.indexing.min_segment_duration == 1.5
30          assert cfg.sport == "tennis"
31
32
33      def test_apply_overrides_rejects_bad_format():
34          cfg = _fresh_config()
35          with pytest.raises(ValueError):
36              wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39      def test_write_intervals_csv(tmp_path):
40          segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
41                  {"start_time": 9.0, "end_time": 12.5}]
42          out = tmp_path / "intervals.csv"
43          wmain._write_intervals_csv(segs, str(out))
```

```
        lines = out.read_text().splitlines()
        assert lines[0] == "start,end,shot_count,ending_reason"
        assert lines[1].startswith("2.000,5.000,4,")
        assert lines[2].startswith("9.000,12.500,,")


    def test_write_report_json(tmp_path):
        out = tmp_path / "report.json"
        wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
        data = json.loads(out.read_text())
        assert data["video_id"] == "vid1" and data["segment_count"] == 1
        assert data["segments"][0] == {"start": 1.0, "end": 2.0}


    def test_parser_infer_accepts_set_and_uris():
        args = wmain.build_parser().parse_args([
            "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
            "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
        assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
        assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


    # ------------------------------------------------------------- cmd_infer (local,
mocked)

    class _OKResult:
        passed = True
        validator_name = "fake"
        message = "ok"

        def model_dump(self):
            return {"validator_name": "fake", "passed": True, "message": "ok"}


    class _FailResult(_OKResult):
        passed = False
        message = "too blurry"


    class _FakeSeg:
        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return [
                {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
                {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
            ]


    def _fresh_config():
        # Load defaults without touching any cwd config.json.
        import tempfile
        p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
        p.write("{}")
        p.close()
        return load_config(p.name)


    @pytest.fixture
    def mocked_pipeline(monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
```

```
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_OKResult()], {}))
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


    def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        out = tmp_path / "bucket"
        scratch = tmp_path / "scratch"

        rc = wmain.main([
            "infer", "--video-uri", str(video), "--out-uri", str(out),
            "--config", str(cfg), "--scratch", str(scratch),
            "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
        ])
        assert rc == 0
        intervals = out / "outputs" / "vid123" / "intervals.csv"
        report = out / "outputs" / "vid123" / "report.json"
        assert intervals.exists() and report.exists()
        rows = intervals.read_text().splitlines()
        assert len(rows) == 3                          # header + 2 rallies
        assert json.loads(report.read_text())["segment_count"] == 2


    class _EmptySeg:
        """A segmenter that finds nothing ? the cold-start failure mode."""

        def __init__(self, db, config):
            pass

        def process_video(self, video_id, path, max_frames=None):
            return []


    def test_setup_logging_levels():
        import logging
        wmain._setup_logging(False)
        assert logging.getLogger().level == logging.INFO
        wmain._setup_logging(True)
        assert logging.getLogger().level == logging.DEBUG


    def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
        # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_OKResult()], {}))
        monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKE")
        out = tmp_path / "bucket"

        # Call cmd_infer directly (not via main(), whose _setup_logging force-reconfigures
handlers and
        # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
        args = wmain.build_parser().parse_args([
            "infer", "--video-uri", str(video), "--out-uri", str(out),
```

```
            "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"])
        with caplog.at_level("WARNING", logger="backend.worker"):
            rc = wmain.cmd_infer(args)
        assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
        msgs = " ".join(r.getMessage() for r in caplog.records)
        assert "0 segments" in msgs and "vidZ" in msgs
        assert "detector_impl=native" in msgs       # surfaces the resolved detector
        assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
        assert json.loads((out / "outputs" / "vidZ" /
"report.json").read_text())["segment_count"] == 0


    def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
        monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_FailResult()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"X")
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")
        rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
"o"),
                         "--config", str(cfg), "--scratch", str(tmp_path / "s")])
        assert rc == 2


    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---


    def test_cmd_train_orchestration(tmp_path, monkeypatch):
        import os
        import subprocess

        labels_dir = tmp_path / "corpus" / "labels"
        labels_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
                         "--version", "0.0.1", "--config", str(cfg), "--scratch",
str(scratch)])
        assert rc == 0
        # bundle pushed to <out-uri>/weights/<version>/
        assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
        assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
```

```
        # the manifest fed to the harness enumerated both labeled videos with a trajectory
each
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2
        assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


    def test_parser_train_detector_args():
        args = wmain.build_parser().parse_args([
            "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
"1.0.0",
            "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
        assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"


    def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
        """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
With
        RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
runner)."""
        import os
        import subprocess

        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no
GPU/WSL)
        # Fake bytes aren't decodable, so stub the (strict, fail-loud) ffprobe probe to a
real value.
        monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
        corpus = tmp_path / "corpus"
        labels_dir = corpus / "labels"
        videos_dir = corpus / "videos"
        labels_dir.mkdir(parents=True)
        videos_dir.mkdir(parents=True)
        header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
        (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
        (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
        (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
        (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
        (videos_dir / "v1.srt").write_text("1\n00:00:00,000 --> 00:00:01,000\nx\n")  #
sidecar: must NOT shadow v1.mp4
        out_dir = tmp_path / "out"
        scratch = tmp_path / "scratch"
        cfg = tmp_path / "config.json"
        cfg.write_text("{}")

        def fake_run(cmd, *a, **k):
            out = cmd[cmd.index("--out") + 1]
            ver = cmd[cmd.index("--version") + 1]
            bdir = os.path.join(out, ver)
            os.makedirs(bdir, exist_ok=True)
            for fn in ("weights.json", "manifest.json"):
                with open(os.path.join(bdir, fn), "w") as f:
                    f.write("{}")
            return subprocess.CompletedProcess(cmd, 0)
        monkeypatch.setattr(subprocess, "run", fake_run)

        rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                         "--version", "0.0.2", "--config", str(cfg), "--scratch",
str(scratch),
                         "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
        assert rc == 0
        specs = json.loads((scratch / "manifest.json").read_text())
        assert len(specs) == 2  # the v1.srt sidecar did NOT shadow v1.mp4 (extension
filter)
        # Real-extract path: each spec points at an on-disk trajectory the runner actually
```

```
wrote.
282         assert all(os.path.exists(s["trajectory"]) for s in specs)
283         assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub
runner's output
284
285
286     def test_cmd_train_detector_native_on_tracknet_fails_clean_not_traceback(tmp_path,
monkeypatch):
287         """detector_impl=native for a family with no native runner must return rc 2 (clean),
not
288         propagate a raw NotImplementedError traceback out of the worker."""
289         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
290         corpus = tmp_path / "corpus"
291         labels_dir = corpus / "labels"
292         (labels_dir).mkdir(parents=True)
293         (corpus / "videos").mkdir(parents=True)
294         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
295         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
296         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
297         cfg = tmp_path / "config.json"
298         cfg.write_text("{}")
299         rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path /
"out"),
300                          "--version", "0.0.3", "--config", str(cfg), "--scratch",
str(tmp_path / "s"),
301                          "--trajectory-source", "detector", "--segmenter",
"tracknet_hybrid"])
302         assert rc == 2  # graceful: _resolve_train_detector caught NotImplementedError,
returned None
303
304
305     def test_probe_video_fps_width_returns_none_on_unreadable(tmp_path):
306         """The strict ffprobe probe must return None (caller skips) for a non-video file,
never guess."""
307         bad = tmp_path / "not_a_video.mp4"
308         bad.write_bytes(b"NOTAVIDEO")
309         assert wmain._probe_video_fps_width(str(bad)) is None
310
```

13. assistant (2026-06-22T12:13:49.358Z)

I now have everything I need. The bug and fix are exactly as described in `docs/DATA_IN_GCS.md` §7b blocker #1. Let me create the branch off `origin/master` (per branching-safety convention), then apply the fix.

14. user (2026-06-22T12:13:51.051Z)

```
---untracked that matter---
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---create branch from origin/master---
branch 'fix/worker-train-dated-label-join' set up to track 'origin/master'.
Switched to a new branch 'fix/worker-train-dated-label-join'
---verify HEAD---
fix/worker-train-dated-label-join
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
```

15. assistant (2026-06-22T12:13:55.300Z)

Now I'm on a clean branch off `origin/master`. Let me apply the fix. First, add `import re`:

16. user (2026-06-22T12:13:57.034Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

17. assistant (2026-06-22T12:13:59.595Z)

Now add the helper near `_VIDEO_EXTS`:

18. user (2026-06-22T12:14:09.437Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

19. assistant (2026-06-22T12:14:12.046Z)

Now wire the helper into `cmd_train`:

20. user (2026-06-22T12:14:14.348Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

21. assistant (2026-06-22T12:14:17.082Z)

Now add the unit tests for the helper plus a regression test for the dated-label?bare-video join. Let me insert them after the existing train tests:

22. user (2026-06-22T12:14:31.801Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-22T12:14:33.794Z)

Now let me run the targeted worker tests first, then the full pre-LGTM gate.

24. user (2026-06-22T12:14:37.843Z)

```
..........................                                               [100%]
26 passed in 2.16s
```

25. assistant (2026-06-22T12:14:41.046Z)

All 26 worker tests pass. Now the full pre-LGTM gate ? tests with coverage, ruff, and mypy:

26. user (2026-06-22T12:17:17.207Z)

```
backend\results.py                                27      0   100%
backend\sports\__init__.py                        17      0   100%
backend\sports\badminton.py                       13      1    92%
backend\sports\base.py                            12      3    75%
backend\storage\__init__.py                       57      4    93%
backend\storage\base.py                           14      0   100%
backend\storage\gcs.py                            41      4    90%
backend\storage\gdrive.py                         61      3    95%
backend\storage\local.py                          49      3    94%
backend\storage\ref.py                            48      0   100%
backend\storage\s3.py                             47      5    89%
backend\tools\__init__.py                          0      0   100%
backend\tools\export.py                           75      2    97%
backend\tools\rally_slicer.py                    105     47    55%
backend\utils\__init__.py                          0      0   100%
backend\utils\app_home.py                         40      0   100%
backend\utils\ffmpeg.py                           24      0   100%
```

```
backend\utils\geometry.py                              29      0   100%
backend\utils\hashing.py                                9      0   100%
backend\utils\logging_setup.py                         11      0   100%
backend\utils\paths.py                                 33      3    91%
backend\utils\redact.py                                16      0   100%
backend\utils\run_telemetry.py                        126     12    90%
backend\utils\single_instance.py                       43     11    74%
backend\utils\video.py                                 26      3    88%
backend\utils\workspace.py                             49      0   100%
backend\worker\__init__.py                              0      0   100%
backend\worker\__main__.py                            300     37    88%
training\__init__.py                                    0      0   100%
training\gen0\__init__.py                               0      0   100%
training\gen0\harness.py                              129      9    93%
-----------------------------------------------------------------------
TOTAL                                               10001   1449    86%
Required test coverage of 70% reached. Total coverage: 85.51%
========================== short test summary info ==========================
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_
cuts_over_seg
1 failed, 1426 passed, 7 warnings in 152.62s (0:02:32)
=======================================================================
 RESULT: FAIL - pytest exit 1
=======================================================================
```

27. assistant (2026-06-22T12:17:27.634Z)

One unrelated test failed: `test_served_gate_a_regression.py` (gate-A proposal windowing) ? a code path my change doesn't touch. I must confirm it's pre-existing, not caused by me. Let me investigate.

28. user (2026-06-22T12:17:35.532Z)

```
            iv = [(w["start"], w["end"]) for w in wins]
            est = rally_gate.estimate_play_axis(pts)
            gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
            on = [(g.start, g.end) for g in gated if g.crossings >= 2]
            gts = load_golden_gts(s["labels"])
            d_on.append(score(on, gts).f1)
            d_off.append(score(iv, gts).f1)
            seg_on.append(_segm.segment_count_ratio(on, gts))
            seg_off.append(_segm.segment_count_ratio(iv, gts))
        n = len(d_on)
        mean_delta = (sum(d_on) - sum(d_off)) / n
        mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n
        # Durable direction-of-effect (corpus-independent): the gate only drops windows, so
under the
        # over-segmenting proposal windowing it lowers the over-seg ratio.
        assert mean_seg_on < mean_seg_off, (mean_seg_on, mean_seg_off)
        # Honest measured snapshot (2026-06-18, present 9-clip multicourt set): mean ?F1 ?
-0.001,
        # over-seg 1.16 ? 0.64 (was +0.074 / 1.68 ? 1.01 on the retired original-6). ~F1-neutral
band:
        assert mean_delta == pytest.approx(0.0, abs=0.03), mean_delta
>       assert mean_seg_off == pytest.approx(1.16, abs=0.10), mean_seg_off
E       AssertionError: 1.3639692020429994
E       assert 1.3639692020429994 == 1.16 ± 0.1
E
E         comparison failed
E         Obtained: 1.3639692020429994
E         Expected: 1.16 ± 0.1

tests\test_served_gate_a_regression.py:179: AssertionError
========================== short test summary info ==========================
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_
```

```
cuts_over_seg
1 failed in 6.17s
```

29. assistant (2026-06-22T12:17:48.359Z)

```
This failure is a **corpus-snapshot drift**, not my change: it asserts `mean_seg_off == 1.16 ±
0.10` against a local golden fixture set and got `1.36`. My change only touches `cmd_train`'s
label-name derivation ? it cannot affect `rally_gate.gate_windows` / `load_golden_gts`. Let me
prove it's pre-existing by reverting my two files and re-running.
```

30. user (2026-06-22T12:17:58.268Z)

```
=== stash my changes (revert to clean origin/master tree) ===
Saved working directory and index state WIP on fix/worker-train-dated-label-join: d301a7f
fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-engine
(#318)
=== run the failing test on clean origin/master ===
E         comparison failed
E         Obtained: 1.3639692020429994
E         Expected: 1.16 ± 0.1

tests\test_served_gate_a_regression.py:179: AssertionError
========================== short test summary info ==========================
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_
cuts_over_seg
1 failed in 6.46s
=== restore my changes ===

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (054f3749f67e9bc5ac704d3fb6ff22b3851e393d)
```

31. assistant (2026-06-22T12:18:04.977Z)

```
Confirmed pre-existing: the identical failure (`1.3639...`) reproduces on a clean
`origin/master` tree with my changes reverted. It's local golden-corpus snapshot drift (the
`golden-corpus-hygiene` label cleanup changed the fixture set) ? unrelated to and untouched by
my change. Now ruff + mypy:
```

32. user (2026-06-22T12:18:09.757Z)

```
=== ruff ===
help: Rewrite `bold` as a `def`

E731 Do not assign a `lambda` expression, use a `def`
  --> artifacts\landing\make_og.py:24:1
   |
23 | bold = lambda s: font("segoeuib.ttf", s)
24 | reg = lambda s: font("segoeui.ttf", s)
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
25 |
26 | M = 80  # margin
   |
help: Rewrite `reg` as a `def`

Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
=== mypy ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 136 source files
```

33. assistant (2026-06-22T12:18:18.354Z)

```
mypy is clean. ruff's 2 errors are both in `artifacts/landing/make_og.py` ? the **untracked**
`artifacts/` dir (owner/sibling WIP, `?? artifacts/` in the initial status), not my files and
not part of this PR. Let me confirm my changed files are ruff-clean and check whether
`artifacts/` is even tracked/ignored.
```

34. user (2026-06-22T12:18:20.339Z)

```
=== ruff on my changed files only ===
All checks passed!
=== is artifacts/ tracked? ===
(empty above = untracked)
=== is artifacts/ gitignored? ===
NOT ignored
```

35. assistant (2026-06-22T12:18:27.478Z)

```
Both my changed files pass ruff cleanly. The 2 ruff errors are in `artifacts/landing/make_og.py`
? untracked owner WIP that won't exist in CI (not committed), so CI's `ruff check .` is
unaffected. I won't touch that file (sibling/owner WIP).

Let me review the final diff before committing:
```

36. user (2026-06-22T12:18:28.076Z)

```
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 081d8f5..08d8d84 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -20,6 +20,7 @@ import csv
 import json
 import logging
 import os
+import re
 import sys
 import tempfile
 from typing import Any, List
@@ -264,6 +265,29 @@ def cmd_infer(args: argparse.Namespace) -> int:
 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

+#: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
+#: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is
NOT
+#: part of the clip's join key, so it's stripped when deriving the videos/ stem.
+_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
+
+
+def _label_clip_name(fname: str) -> str:
+    """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label filename.
+
+    Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date prefix,
so a
+    canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
+
+    * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-stamped)
+    * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same stem)
+    * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of the
name, kept)
```

```
+    * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in <name>
survive)
+
+    Without the date strip, ``--trajectory-source detector`` can't match a dated label to its
bare
+    video stem, so every clip is skipped and train aborts with ``<2 videos`` (DATA_IN_GCS §7b
#1).
+    """
+    name = fname.replace(".rallies.csv", "").replace(".csv", "")
+    return _LABEL_DATE_PREFIX.sub("", name)
+

 def _probe_video_fps_width(path: str):
     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
@@ -380,8 +404,8 @@ def cmd_train(args: argparse.Namespace) -> int:
     print(f"[worker] trajectory source: {args.trajectory_source}")
     specs: List[dict] = []
     for uri in label_uris:
-        fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
-        name = fname.replace(".rallies.csv", "").replace(".csv", "")
+        fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
+        name = _label_clip_name(fname)                         # -> GX020094 (date prefix
stripped)
         local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
         if args.trajectory_source == "detector":
             vuri = video_by_stem.get(name)
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 642e46a..c48fa02 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -191,6 +191,66 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):

 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

+@pytest.mark.parametrize("fname,expected", [
+    ("2026-06-22_GX020094.rallies.csv", "GX020094"),            # canonical dated label
(DATA_IN_GCS §7.1)
+    ("GX020094.rallies.csv", "GX020094"),                       # legacy bare label -> SAME
clip stem
+    ("GX020094_proxy.rallies.csv", "GX020094_proxy"),           # _proxy is part of the name,
kept
+    ("2026-06-21_mahadevpura_1.rallies.csv", "mahadevpura_1"),  # underscores in <name> survive
+    ("v1.csv", "v1"),                                           # bare .csv suffix
+    ("2026-06-22_GX020094.csv", "GX020094"),                    # dated + bare .csv
+])
+def test_label_clip_name_strips_date_prefix(fname, expected):
+    """A dated label and a bare label must both resolve to the bare videos/<name> stem."""
+    assert wmain._label_clip_name(fname) == expected
+
+
+def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
+    """Regression (DATA_IN_GCS §7b blocker #1): canonical labels carry an ISO-date prefix
+    (labels/<date>_<name>.rallies.csv) while videos are bare (videos/<name>.mp4). The label-
derived
+    clip name must strip the date so the video join succeeds ? otherwise every clip is skipped
and
+    train aborts with <2 videos."""
+    import os
+    import subprocess
+
+    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # CPU stub runner (no GPU/WSL)
+    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
+    corpus = tmp_path / "corpus"
```

```
+    labels_dir = corpus / "labels"
+    videos_dir = corpus / "videos"
+    labels_dir.mkdir(parents=True)
+    videos_dir.mkdir(parents=True)
+    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
+    # canonical DATED labels ...
+    (labels_dir / "2026-06-22_GX020094.rallies.csv").write_text(header +
"1,2.0,5.0,winner,badminton,\n")
+    (labels_dir / "2026-06-21_mahadevpura_1.rallies.csv").write_text(header +
"1,3.0,6.0,winner,badminton,\n")
+    # ... joining to BARE-stem videos
+    (videos_dir / "GX020094.mp4").write_bytes(b"FAKEVIDEO1")
+    (videos_dir / "mahadevpura_1.mp4").write_bytes(b"FAKEVIDEO2")
+    out_dir = tmp_path / "out"
+    scratch = tmp_path / "scratch"
+    cfg = tmp_path / "config.json"
+    cfg.write_text("{}")
+
+    def fake_run(cmd, *a, **k):
+        out = cmd[cmd.index("--out") + 1]
+        ver = cmd[cmd.index("--version") + 1]
+        bdir = os.path.join(out, ver)
+        os.makedirs(bdir, exist_ok=True)
+        for fn in ("weights.json", "manifest.json"):
+            with open(os.path.join(bdir, fn), "w") as f:
+                f.write("{}")
+        return subprocess.CompletedProcess(cmd, 0)
+    monkeypatch.setattr(subprocess, "run", fake_run)
+
+    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
+                     "--version", "0.0.4", "--config", str(cfg), "--scratch", str(scratch),
+                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
+    assert rc == 0  # before the fix: every clip skipped -> rc 2 ("<2 videos")
+    specs = json.loads((scratch / "manifest.json").read_text())
+    assert len(specs) == 2                                          # both dated labels joined
+    assert {s["name"] for s in specs} == {"GX020094", "mahadevpura_1"}  # downstream name is
date-stripped
+
+
 def test_cmd_train_orchestration(tmp_path, monkeypatch):
     import os
     import subprocess
```

37. assistant (2026-06-22T12:18:34.493Z)

The diff is clean. Let me verify the new regression test actually *catches* the bug ? revert only the source fix (keep the new test) and confirm the test fails without the fix:

38. user (2026-06-22T12:18:41.633Z)

```
=== revert ONLY the source fix, keep the new test ===
Saved working directory and index state WIP on fix/worker-train-dated-label-join: d301a7f
fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-engine
(#318)
=== run new regression test against UNFIXED source (expect FAIL) ===
E       AttributeError: module 'backend.worker.__main__' has no attribute '_label_clip_name'

tests\test_worker.py:204: AttributeError
========================== short test summary info ==========================
FAILED tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos
FAILED
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.rallies.csv-
GX020094]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094.rallies.csv-
GX020094]
```

```
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094_proxy.rallies.csv-
GX020094_proxy]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-
21_mahadevpura_1.rallies.csv-mahadevpura_1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[v1.csv-v1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.csv-
GX020094]
7 failed in 2.45s
=== restore the fix ===
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (4c9bed8282fa8e721bcbe5a4243621c22b3ff302)
```

39. assistant (2026-06-22T12:19:03.721Z)

```
The new tests fail without the fix (helper tests `AttributeError`; the regression test hits rc
2) and pass with it ? a genuine regression guard.

**Pre-LGTM gate summary:**
- ? Full suite: 1426 passed; 1 failure (`test_served_gate_a_regression`) **proven pre-existing**
on clean `origin/master` ? local golden-corpus snapshot drift, untouched by this change.
- ? Coverage 85.51% ? 70% floor (`backend/worker/__main__.py` at 88%).
- ? ruff: my files clean; the 2 errors are in untracked owner WIP (`artifacts/`), absent from
CI.
- ? mypy: clean (136 files).

Now commit (explicit paths only, per branching-safety) and open the PR:
```

40. user (2026-06-22T12:19:05.258Z)

```
=== staged (should be ONLY my 2 files) ===
backend/worker/__main__.py
tests/test_worker.py
=== re-verify HEAD is mine ===
fix/worker-train-dated-label-join
```

41. assistant (2026-06-22T12:19:09.930Z)

```
Only my 2 files staged, branch cleanly based on `origin/master`. Committing:
```

42. user (2026-06-22T12:19:16.947Z)

```
[fix/worker-train-dated-label-join a2c8be0] fix(worker): strip ISO-date prefix when joining
labels to videos in train
 2 files changed, 86 insertions(+), 2 deletions(-)
=== committed ===
a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train
```

43. assistant (2026-06-22T12:19:19.732Z)

```
Now push and open the PR:
```

44. user (2026-06-22T12:19:22.598Z)

```
remote:
remote: Create a pull request for 'fix/worker-train-dated-label-join' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/worker-train-
dated-label-join
remote:
branch 'fix/worker-train-dated-label-join' set up to track 'origin/fix/worker-train-dated-label-
join'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join
```

45. user (2026-06-22T12:19:42.680Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/319

46. assistant (2026-06-22T12:20:12.768Z)

Done. PR opened: **[badminton-highlight-indexer#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)**

What I did

Fixed `cmd_train`'s label?video join so it works under the canonical GCS naming `labels/<YYYY-MM-DD>_<name>.rallies.csv` (the `docs/DATA_IN_GCS.md` §7b blocker #1).

The fix ? factored the clip-name derivation into a tested helper in backend/worker/__main__.py:

```python
_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")

def _label_clip_name(fname: str) -> str:
    name = fname.replace(".rallies.csv", "").replace(".csv", "")
    return _LABEL_DATE_PREFIX.sub("", name)
```

`cmd_train` now calls `name = _label_clip_name(fname)`, so `2026-06-22_GX020094.rallies.csv` ? `GX020094` and joins to `videos/GX020094.mp4`. `_proxy` and underscores inside `<name>` are preserved; legacy bare labels still map to the same stem. The downstream spec `name` is the clean, date-stripped name. `import re` added at module level.

Tests (both fail without the fix, pass with it):
- `test_label_clip_name_strips_date_prefix` ? parametrized over dated / bare / `_proxy` / underscored / bare-`.csv` cases.
- `test_cmd_train_detector_joins_dated_labels_to_bare_videos` ? end-to-end `--trajectory-source detector` regression: dated labels ? bare-stem videos ? rc 0, 2 specs.

Verification (repo pre-LGTM gate)

Check	Result
`run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`	**1426 passed**, coverage **85.51%** (worker module 88%)
`ruff check` (changed files)	clean
`mypy backend training`	clean (136 files)

One honest caveat: the full suite shows **1 failure**, `test_served_gate_a_regression::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg` (asserts `mean_seg_off == 1.16 ± 0.10`, got `1.36`). I verified it's **pre-existing** ? it reproduces identically on a clean `origin/master` tree with my changes stashed. It's local golden-corpus snapshot drift (the in-flight `golden-corpus-hygiene` label cleanup changed the fixture set), on a code path this PR never touches. The repo-wide ruff errors are likewise in untracked owner WIP under `artifacts/`, which won't exist in CI.

Branched off `origin/master`, staged explicit paths only, one PR for this item. I left `docs/DATA_IN_GCS.md` §7b untouched ? that doc lives on the `work/golden-corpus-hygiene` branch, not master, so flipping blocker #1 to resolved (referencing PR #319) belongs with that branch's merge, not here.

47. user (2026-06-22T12:20:15.099Z)

can you also do a doc fix related to this and merge the PRs once the coverage has been uphelp/improved and the docs fixed

48. assistant (2026-06-22T12:20:55.114Z)

Let me gather the facts I need before deciding where the doc fix goes and how to handle the merge ? CI status, whether the doc even exists on master, whether there's a PR for the corpus branch, and exactly what's uncovered in the worker module.

49. user (2026-06-22T12:21:00.594Z)


```
=== PR #319 CI / mergeable state ===
{"baseRefName":"master","headRefName":"fix/worker-train-dated-label-join","mergeStateStatus":"UN
STABLE","mergeable":"MERGEABLE","statusCheckRollup":[{"__typename":"CheckRun","completedAt":"202
6-06-22T12:19:59Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391288","name":"Ruff lint","startedAt":"2026
-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-06-
22T12:19:56Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391282","name":"Leak scan (PII / attribution
guard)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:18Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391389","name":"Mypy (lenient baseline)","st
artedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:12Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391575","name":"Syntax sweep + import smoke
(no GPU stack)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
0001-01-01T00:00:00Z","conclusion":"","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391441","name":"Full test suite (offline, mo
cked)","startedAt":"2026-06-
22T12:19:50Z","status":"IN_PROGRESS","workflowName":"CI"},{"__typename":"CheckRun","completedAt"
:"2026-06-
22T12:20:14Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391421","name":"Golden fixtures cross-OS
(ubuntu-latest)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
0001-01-01T00:00:00Z","conclusion":"","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391318","name":"Golden fixtures cross-OS
(windows-latest)","startedAt":"2026-06-
22T12:19:51Z","status":"IN_PROGRESS","workflowName":"CI"},{"__typename":"CheckRun","completedAt"
:"2026-06-
22T12:20:14Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391409","name":"Golden fixtures cross-OS
(macos-latest)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:21Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391301","name":"Dependency license scan (no
copyleft in runtime deps)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:58Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391443","name":"Built wheel smoke (install +
launch)","startedAt":"2026-06-22T12:19:50Z","status":"COMPLETED","workflowName":"CI"}]}

=== does DATA_IN_GCS.md exist on origin/master? ===
fatal: path 'docs/DATA_IN_GCS.md' does not exist in 'origin/master'
NO ? not on master

=== which branches have the doc? ===
  work/golden-corpus-hygiene
  remotes/origin/work/golden-corpus-hygiene

=== open PRs ===
```

[{"baseRefName":"master","headRefName":"feat/nightly-scheduler-fix","number":320,"title":"feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger"},{"baseRefName":"master","headRefName":"fix/worker-train-dated-label-join","number":319,"title":"fix(worker): strip ISO-date prefix when joining labels to videos in train"},{"baseRefName":"master","headRefName":"work/golden-corpus-hygiene","number":316,"title":"docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration"}]

50. assistant (2026-06-22T12:22:14.204Z)

Key findings that shape the plan:

- **PR #319 CI**: nearly all green (ruff, mypy, leak scan, license scan, wheel smoke, golden-fixtures macOS/Ubuntu all ?); "Full test suite" + "Golden fixtures windows-latest" still **in progress**. `mergeable: MERGEABLE`.
- **`docs/DATA_IN_GCS.md` is NOT on master** ? it lives only on `work/golden-corpus-hygiene`, which is **open PR [#316](https://github.com/Khelsutra/badminton-highlight-indexer/pull/316)**. So the doc fix belongs on #316's branch, not #319.
- "The PRs" = **#319 (code fix)** + **#316 (doc map)**, both this session's track. #320 is an unrelated nightly-scheduler item.

So: improve coverage on #319, add the resolved-blocker doc fix to #316, then merge both once CI is green. Let me get the authoritative coverage gaps for the worker module and re-read the doc tracker sections in parallel.

51. user (2026-06-22T12:22:15.069Z)

Command running in background with ID: b51kt55u4. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\b51kt55u4.output. You will be notified when it completes. To check interim output, use Read on that file path.

52. assistant (2026-06-22T12:22:25.473Z)

While coverage runs, let me confirm the working-tree state of the doc (it shouldn't exist on my current branch since it's not on master) and read the exact tracker lines from the corpus branch via git:

53. user (2026-06-22T12:22:28.866Z)

```
=== does the doc exist in my current working tree? ===
absent (expected ? not on this branch)

=== current branch ===
fix/worker-train-dated-label-join

=== PR #316 head sha + is its branch ff-able / behind master? ===
4       4
(left=master-ahead  right=branch-ahead)

=== any uncommitted tracked changes right now? ===
(none ? clean tracked tree)
```

54. assistant (2026-06-22T12:22:37.543Z)

Coverage output is ready. Let me read it and check PR #316's mergeable state (it's diverged ? 4 ahead / 4 behind master):

55. user (2026-06-22T12:22:38.217Z)

```
1       ........................                                 [100%]
2       ============================== tests coverage ==============================
3       _________________ coverage: platform win32, python 3.13.13-final-0 _________________
4
```

```
5       Name                         Stmts   Miss  Cover   Missing
6       -------------------------------------------------------
7       backend\worker\__init__.py       0      0   100%
8       backend\worker\__main__.py     300     55    82%   114-115, 126-139, 146-155, 162, 170,
204-206, 211-212, 260, 309-314, 333-335, 338-340, 350-351, 372, 386-388, 413-414, 422-423,
427-428, 439-440, 451-453, 457-458, 542
9       -------------------------------------------------------
10      TOTAL                          300     55    82%
11      26 passed in 3.16s
12
```

56. assistant (2026-06-22T12:23:04.341Z)

82% from worker tests alone (55 missed); the full-suite figure is 88%. The most valuable, directly-related uncovered branch is the **"no video for `{name}` ? skipping"** path (the exact failure mode my fix addresses). Let me read the current `cmd_train` region to map the uncovered lines precisely.

57. user (2026-06-22T12:23:04.989Z)

```
400                 if not vname.lower().endswith(_VIDEO_EXTS):
401                     continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
402                 video_by_stem[os.path.splitext(vname)[0]] = vuri
403
404         print(f"[worker] trajectory source: {args.trajectory_source}")
405         specs: List[dict] = []
406         for uri in label_uris:
407             fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
408             name = _label_clip_name(fname)                         # -> GX020094 (date
prefix stripped)
409             local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
410             if args.trajectory_source == "detector":
411                 vuri = video_by_stem.get(name)
412                 if not vuri:
413                     print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
414                     continue
415                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
416                                             config=storage_cfg)
417                 video_id = compute_video_id(local_video)
418                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
419                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
420                 meta = _probe_video_fps_width(local_video)
421                 if meta is None:
422                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
423                     continue
424                 fps, frame_width = meta
425                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
426                 if not traj:
427                     print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
428                     continue
429                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
430                               "fps": fps, "frame_width": frame_width})
431             else:
432                 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
433                 traj = os.path.join(traj_dir, f"{name}_traj.csv")
```

```
434                 synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
435                 specs.append({"name": name, "trajectory": traj, "labels": local_label,
436                               "fps": args.fps, "frame_width": args.frame_width})
437
438         if len(specs) < 2:
439             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
440             return 2
441         manifest_path = os.path.join(scratch, "manifest.json")
442         with open(manifest_path, "w", encoding="utf-8") as f:
443             json.dump(specs, f, indent=2)
444         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
```

58. assistant (2026-06-22T12:24:49.617Z)

I have the exact uncovered branches. Beyond my fix, the most valuable, in-scope coverage gains are `cmd_train`'s per-label skip branches (lines 412-414 join-miss ? the exact failure mode I fixed ? plus 421-423 probe-fail and 426-428 no-trajectory). Let me add two focused tests that exercise them.

59. user (2026-06-22T12:25:08.638Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

60. assistant (2026-06-22T12:25:11.444Z)

Now run the worker tests and confirm coverage improved:

61. user (2026-06-22T12:25:17.960Z)

```
.............................                                                [100%]
=============================== tests coverage ================================
_________________ coverage: platform win32, python 3.13.13-final-0 _________________

Name                           Stmts   Miss  Cover   Missing
--------------------------------------------------------------
backend\worker\__init__.py         0      0   100%
backend\worker\__main__.py       300     49    84%   114-115, 126-139, 146-155, 162, 170, 204-206,
211-212, 260, 309-314, 333-335, 338-340, 350-351, 372, 386-388, 439-440, 451-453, 457-458, 542
--------------------------------------------------------------
TOTAL                            300     49    84%
28 passed in 3.47s
```

62. assistant (2026-06-22T12:25:41.683Z)

Coverage improved 82% ? 84% (lines 412-414, 421-423, 426-428 now covered; 28 tests pass). One more cheap, thematically-perfect win: line 438-440 is the literal `<2 videos` abort the bug tripped for *every* clip ? let me cover it via a legitimate single-join case:

63. user (2026-06-22T12:25:53.932Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

64. user (2026-06-22T12:26:01.399Z)

```
.............................                                                [100%]
=============================== tests coverage ================================
_________________ coverage: platform win32, python 3.13.13-final-0 _________________

Name                           Stmts   Miss  Cover   Missing
```

```
------------------------------------------------------------
backend\worker\__init__.py       0      0   100%
backend\worker\__main__.py     300     47    84%   114-115, 126-139, 146-155, 162, 170, 204-206,
211-212, 260, 309-314, 333-335, 338-340, 350-351, 372, 386-388, 451-453, 457-458, 542
------------------------------------------------------------
TOTAL                          300     47    84%
29 passed in 3.23s
```

65. assistant (2026-06-22T12:26:18.147Z)

Now 29 tests pass; worker coverage 47 missed (84%, up from 82%) ? the join-miss, probe-fail, no-trajectory, and `<2 videos` branches are all covered now. Let me commit these test additions to #319 and run the full pre-LGTM gate to confirm overall coverage holds.

66. user (2026-06-22T12:26:22.435Z)

```
=== staged ===
tests/test_worker.py
=== verify HEAD is mine ===
fix/worker-train-dated-label-join
a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train
[fix/worker-train-dated-label-join b3b4952] test(worker): cover cmd_train per-label join/skip
branches
 1 file changed, 131 insertions(+)
=== committed ===
b3b4952 test(worker): cover cmd_train per-label join/skip branches
a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train
```

67. assistant (2026-06-22T12:26:25.619Z)

Now run the full pre-LGTM gate locally to confirm overall coverage holds and no regressions, then push:

68. user (2026-06-22T12:28:59.375Z)

```
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   a2c8be0..b3b4952  fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join
=== full suite + coverage (the gate) ===
TOTAL                                                10001   1441    86%
Required test coverage of 70% reached. Total coverage: 85.59%
========================== short test summary info ==========================
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_
cuts_over_seg
1 failed, 1429 passed, 7 warnings in 147.40s (0:02:27)
============================================================================
 RESULT: FAIL - pytest exit 1
============================================================================
```

69. assistant (2026-06-22T12:29:16.822Z)

Full suite: **1429 passed**, overall coverage **85.59%** (up from 85.51%), only the same pre-existing `test_served_gate_a_regression` failure. Critical question: does that test run/fail in CI too? It depends on local golden-corpus presence. Let me check PR #319's actual CI result and the test's skip condition.

70. user (2026-06-22T12:29:22.054Z)

```
=== PR #319 full-suite CI job result ===
{"conclusion":"SUCCESS","name":"Full test suite (offline, mocked)","status":"COMPLETED"}

=== overall mergeStateStatus ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}

=== how does test_served_gate_a_regression source its corpus? (skip guard?) ===
```

```
21:On that corpus the served path is **recall-starved**: WASB tracks a single shuttle while
several
24:windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
28:The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the *original-6*
corpus
31:**present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
33:re-measurement as the corpus grows.
43:# (CI), skip the whole module rather than erroring at import.
44:pytest.importorskip("cv2")
53:# The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh
57:MANIFEST = os.environ.get(
59:    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
```

71. assistant (2026-06-22T12:29:32.128Z)

Confirmed: PR #319's "Full test suite" CI job = **SUCCESS**. The `test_served_gate_a_regression` failure is local-only ? the test sources `output/human_lovo_manifest.json` (gitignored, **absent in CI** ? skipped), so CI never sees the local corpus drift. This fully validates that the local failure is environmental, not a real regression.

Let me confirm the complete CI rollup has zero failures (and why it shows UNSTABLE):

72. user (2026-06-22T12:29:36.218Z)

```
=== ALL checks for #319 (name | status | conclusion) ===
      1 FAILURE Ruff lint
      1 SUCCESS Built wheel smoke (install + launch)
      1 SUCCESS Dependency license scan (no copyleft in runtime deps)
      1 SUCCESS Full test suite (offline, mocked)
      1 SUCCESS Golden fixtures cross-OS (macos-latest)
      1 SUCCESS Golden fixtures cross-OS (ubuntu-latest)
      1 SUCCESS Golden fixtures cross-OS (windows-latest)
      1 SUCCESS Leak scan (PII / attribution guard)
      1 SUCCESS Mypy (lenient baseline)
      1 SUCCESS Syntax sweep + import smoke (no GPU stack)

=== any non-SUCCESS? ===
FAILURE Ruff lint
```

73. assistant (2026-06-22T12:29:47.180Z)

CI now flags **Ruff lint = FAILURE** on my latest push ? my new test code must have a violation (CI only sees committed files; `artifacts/` is untracked). Let me reproduce exactly:

74. user (2026-06-22T12:29:49.201Z)

```
=== ruff on my test file ===
F401 [*] `os` imported but unused
   --> tests\test_worker.py:359:12
    |
357 |     This is the guard the date-prefix bug used to trip for EVERY clip (DATA_IN_GCS §7b
#1); here it
358 |     fires legitimately because only one of two labels has a matching video."""
359 |     import os
    |            ^^
360 |     import subprocess
    |
help: Remove unused import: `os`

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

```
=== ruff config the CI uses? ===
16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
28:      - name: Install ruff
29:        run: python -m pip install ruff
31:        run: ruff check . --output-format github
102:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
---
```

75. assistant (2026-06-22T12:29:55.460Z)

Trivial: unused `import os` in my 4th test (it uses `subprocess` but not `os`). Let me remove it:

76. user (2026-06-22T12:29:57.804Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

77. user (2026-06-22T12:30:06.237Z)

```
=== ruff (whole repo minus untracked artifacts) on tracked files ===
All checks passed!
=== re-run worker tests to confirm still green after removing import ===
.............................                                            [100%]
29 passed in 2.10s
```

78. assistant (2026-06-22T12:30:19.747Z)

Ruff clean, 29 tests pass. Since this PR will squash-merge (per the repo's merge style), I'll add a small follow-up commit rather than force-push:

79. user (2026-06-22T12:30:24.298Z)

```
[fix/worker-train-dated-label-join 56c1415] style(worker-test): drop unused os import (ruff
F401)
 1 file changed, 1 deletion(-)
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   b3b4952..56c1415  fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join
```

80. assistant (2026-06-22T12:30:34.258Z)

PR #319 updated; CI will re-run green. Now the doc fix on PR #316's branch. Let me switch to `work/golden-corpus-hygiene` carefully (shared-checkout safety): fetch, verify local matches origin, `--ff-only` pull.

81. user (2026-06-22T12:30:35.270Z)

Chapter marked: "Doc fix on corpus branch". Continue your current work.

82. user (2026-06-22T12:30:44.633Z)

```
=== local vs origin for golden-corpus-hygiene (should be 0 0) ===
fatal: ambiguous argument 'work/golden-corpus-hygiene...origin/work/golden-corpus-hygiene':
unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
=== switch ===
error: pathspec 'work/golden-corpus-hygiene' did not match any file(s) known to git
=== ff-only pull ===
Already up to date.
=== confirm doc present + HEAD ===
56c1415 style(worker-test): drop unused os import (ruff F401)
```

83. assistant (2026-06-22T12:30:53.503Z)

The local branch isn't matching ? let me list branches to see the actual state (shared checkout may have shifted):

84. user (2026-06-22T12:30:55.076Z)

```
Exit code 128
=== local branches ===
+ bhi-execpolicy
  chore/consolidate-fusion-schema-predicate
  docs/adr-012-gpu-cloudrun
  docs/archive-decoupled-compute
  docs/archive-decoupled-v2
  docs/compute-decoupled-serving
  docs/court-floor-rally-end
  docs/decouple-scope-defer
  docs/deploy-report-polish
  docs/desktop-app-plan
  docs/doc-status-register
  docs/drop-digitalocean-gcp-only
  docs/findings-1-2-pervideo-nextsteps
  docs/gcp-deploy-report
  docs/gcp-pivot-runbook
  docs/lock-m0-decisions
+ docs/lovo-resweep-n13
  docs/m0-approved-d13-d15
  docs/m2-finalized-report
  docs/packaging-plan-tracked
  docs/pkg-p2c-done
  docs/point-i18n-ui-to-khelsutra
+ docs/rally-gap-closure
+ docs/recording-single-match
  docs/restructure-field-roora-data
  docs/serving-design-picks-p3-reconcile
  feat/api-m1-health-downloadurl
  feat/gcp-ops-agent-and-dryrun-guide
  feat/gdrive-storage-backend
  feat/highlights-runner
  feat/local-no-ai-segmenter
  feat/locale-analytics
  feat/localized-watermark
  feat/long-rally-lens
  feat/native-wasb-runner
  feat/packaging-p0a-app-home
  feat/packaging-p0b-config-redact
  feat/packaging-p0c-bind-host
  feat/packaging-p0d-scipy-ci
  feat/packaging-p0e-single-instance
  feat/packaging-p1a-endpoints
  feat/packaging-p1b-bundle-ui
  feat/packaging-p2a-ffmpeg
  feat/packaging-p2b-launcher
  feat/packaging-p2d-db-safety
  feat/packaging-p2e-lazy-torch
  feat/pkg-bundle-p2c-wizard
  feat/pkg-p2b-install
  feat/prominent-court-gate
  feat/serving-m1-deployment-profile
  feat/serving-m2-input-backend
  feat/serving-m3-output-base
  feat/serving-m4-device-context
  feat/serving-m5-truly-local-checks
  feat/serving-m6-cloudrun-dispatch
  feat/serving-m8-cost-ledger
```

```
+ feat/w1-hard-cap-fallback
+ feat/w2-never-empty-safeguard
  fix/compile-resolve-bucket-source
  fix/compiler-dedup-overlapping-segments
  fix/dryrun-guide-font-and-ui-tracking
  fix/engine-double-module-job-hang
  fix/finding-b-loud-unrunnable-config
  fix/fusion-loaders-mixed-schema
  fix/gcp-teardown-default-delete
  fix/gemini-loud-chunk-failures
  fix/gemini-per-video-chunk-dir
  fix/infer-observability
+ fix/runner-stitch-watermark
  fix/served-gate-a-corpus-regrounding
  fix/wasb-test-mocks-readability-preflight
* fix/worker-train-dated-label-join
  fix/wsl-unreadable-source-preflight
+ master
  ops/sync-master-d749fb9
+ perf/wasb-fast-frames
+ run/boxhill17-highlights
  serving/exposure-safety-preflight
  worktree-agent-a771ff1d85e18f2ed
  worktree-wf_1d39e81c-851-1
  worktree-wf_1d39e81c-851-3
  worktree-wf_1d39e81c-851-4
  worktree-wf_1d39e81c-851-5
  worktree-wf_1d39e81c-851-6
  worktree-wf_1d39e81c-851-7
  worktree-wf_87c49086-be4-1
  worktree-wf_87c49086-be4-2
  worktree-wf_87c49086-be4-3
  worktree-wf_87c49086-be4-4
  worktree-wf_87c49086-be4-5
  worktree-wf_87c49086-be4-6
  worktree-wf_87c49086-be4-7
=== remote golden-corpus-hygiene exists? ===
=== origin/work/golden-corpus-hygiene HEAD ===
fatal: ambiguous argument 'origin/work/golden-corpus-hygiene': unknown revision or path not in
the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
```

85. assistant (2026-06-22T12:31:07.965Z)

The `work/golden-corpus-hygiene` branch has **vanished from local and remote** ? a sibling session or the owner likely merged/closed PR #316 in the last few minutes (classic shared-checkout shift). Let me check #316's state and whether the doc is now on master:

86. user (2026-06-22T12:31:12.742Z)

```
=== PR #316 state ===
{"headRefName":"work/golden-corpus-hygiene","mergeCommit":"e221cb928ca1595102fbf7ad1bc0e64727bac
b35","mergedAt":"2026-06-22T12:29:47Z","state":"MERGED"}
=== is DATA_IN_GCS.md on origin/master NOW? ===
YES ? now on master
=== origin/master HEAD now ===
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
```

87. assistant (2026-06-22T12:31:37.531Z)

Now it's clear: **PR #316 merged** (12:29Z, commit e221cb9) and **PR #320 merged** too ? so `docs/DATA_IN_GCS.md` is **now on master**, and master advanced d301a7f ? 48f9d20. This actually simplifies things: the doc fix becomes its own PR off the updated master (matching the repo's code-vs-docs PR separation, and the user's "merge the PRs" plural). Let me verify #319 is still mergeable against new master, then branch for the doc fix.

88. user (2026-06-22T12:31:40.857Z)

```
=== #319 still mergeable vs new master? ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
=== create doc branch off origin/master (has the doc now) ===
branch 'docs/data-gcs-train-blocker-resolved' set up to track 'origin/master'.
Switched to a new branch 'docs/data-gcs-train-blocker-resolved'
=== verify HEAD + doc present ===
docs/data-gcs-train-blocker-resolved
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
doc present
```

89. assistant (2026-06-22T12:32:04.018Z)

I'm on a clean doc branch off the updated master, doc present. Let me read the exact current §7b and §8 text to craft a precise, honest edit:

90. user (2026-06-22T12:32:04.941Z)

```
202     ---
203
204     ## 7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)
205
206     1. **Dated labels break `worker train --trajectory-source detector` (code-verified).**
`cmd_train`
207        derives the join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`,
then looks up
208        `video_by_stem.get(name)` against bare video stems (`GX020094`) ? so every clip is
skipped and train
209        aborts with `<2 videos`
([backend/worker/__main__.py:384](../backend/worker/__main__.py):384 vs
210        :378/:387). **Fix (recommended):** strip a leading `^\d{4}-\d{2}-\d{2}_` from the
label-derived
211        `name` in `cmd_train` (a ~1-line change, own PR + test) ? preserves the locked human-
readable naming
212        AND unblocks detector-train. *(`synth` train + all eval are unaffected ? they don't
join videos.)*
213     2. **Phase-3 eval-on-GCS is net-new plumbing, not a flag** ? see §6 Phase-3. Eval is
local-FS only.
214     3. **`manifests/golden.json` is consumed by zero code today** (train ignores it; eval
can't fetch
215        `gs://`) ? it's correct-but-dormant until blocker #2 lands. Add a tiny manifest-
validator + a CI job
216        that authenticates to GCS and runs the litmus against it, or the DoD ("CI reads only
from `gs://`")
217        stays unverifiable.
218     4. **Gating infra (owner-only, not done):** GPU `GPUS_ALL_REGIONS` quota is still **0**
(no VM ever
219        created ? `deploy/gcp/README` §2); cloud-box secrets (SA JSON + rotated
`GEMINI_API_KEY` + WASB
220        weights staged from `models/`) aren't set up persistently. No real cloud train/infer
runs until these.
221     5. **Reproducibility:** the migration + copy helpers live in gitignored `scratch/`
222        (`migrate_golden_to_gcs.py`, `copy_videos_to_gcs.ps1`) ? promote to `scripts/` so
Phase-2/re-syncs
223        are repeatable from the repo, and add a referenced **proxy-generation** step for the
8 large F: originals.
224
```

```
225     ## 8. Progress tracker
226
227     | Item | Status |
228     |---|---|
229     | Bucket + SA + provisioning (deploy/gcp) | ? built |
230     | Cloud worker `infer`/`train` by URI | ? built (`backend/worker`) |
231     | This data-map doc | ? 2026-06-22 |
232     | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
233     | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
234     | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
235     | Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified
dupes, deleted) |
236     | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
**1/15** demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the
owner-box script |
237     | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
238     | Phase 3: eval/litmus default to the GCS manifest | ? |
239
240     **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
241     reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.
242
243     ---
```

91. assistant (2026-06-22T12:32:30.937Z)

Now I'll make the honest doc edit ? flip §7b blocker #1 to RESOLVED (keeping the lineage/history) with an accurate description of what landed (`_label_clip_name` helper + tests), referencing PR #319, and add a tracker row in §8.

92. user (2026-06-22T12:32:38.464Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA_IN_GCS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

93. user (2026-06-22T12:32:47.038Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA_IN_GCS.md has been updated successfully. (file state is current in your context ? no need to Read it back)

94. assistant (2026-06-22T12:32:52.571Z)

Let me verify the doc change is clean, stage only the doc, and commit:

95. user (2026-06-22T12:32:54.492Z)

```
=== status (only doc should be modified) ===
 M docs/DATA_IN_GCS.md
=== diff stat ===
 docs/DATA_IN_GCS.md | 18 ++++++++++-------
 1 file changed, 11 insertions(+), 7 deletions(-)
=== verify HEAD/base ===
docs/data-gcs-train-blocker-resolved
```

96. assistant (2026-06-22T12:32:57.854Z)

Only the doc is modified, clean base. Commit, push, and open the doc PR:

97. user (2026-06-22T12:33:03.687Z)

```
[docs/data-gcs-train-blocker-resolved 62bcdf4] docs(data): mark DATA_IN_GCS §7b blocker #1
resolved by #319
 1 file changed, 11 insertions(+), 7 deletions(-)
```

```
remote:
branch 'docs/data-gcs-train-blocker-resolved' set up to track 'origin/docs/data-gcs-train-
blocker-resolved'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      docs/data-gcs-train-blocker-resolved -> docs/data-gcs-train-blocker-
resolved
```

98. user (2026-06-22T12:33:14.745Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/321
```

99. assistant (2026-06-22T12:33:50.668Z)

```
Doc PR opened: **[#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321)**.

Before merging (the consequential step), I'll adversarially verify the whole change set ? code
correctness, doc-claim accuracy (no fabricated provenance), and test rigor ? via a focused
verification workflow. The agents read from explicit git refs / PR diffs so they're branch-
independent.
```

100. user (2026-06-22T12:34:19.505Z)

```
Workflow launched in background. Task ID: wt0e0iihs
Summary: Adversarially verify PR #319 (worker train dated-label join fix + tests) and PR #321
(doc resolution) before merge
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-
wf_753ff171-b1d.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-
wf_753ff171-b1d.js"} to iterate without resending the script.)
Run ID: wf_753ff171-b1d
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-
wf_753ff171-b1d.js", resumeFromRunId: "wf_753ff171-b1d"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
```

101. assistant (2026-06-22T12:34:25.585Z)

```
While the verification workflow runs, let me check CI status on both PRs (the ruff fix should
have turned #319 green):
```

102. user (2026-06-22T12:34:29.599Z)

```
=== #319 checks ===
{"checks":["SUCCESS Ruff lint","SUCCESS Leak scan (PII / attribution guard)","SUCCESS Mypy
(lenient baseline)","SUCCESS Syntax sweep + import smoke (no GPU stack)","SUCCESS Full test
suite (offline, mocked)","SUCCESS Golden fixtures cross-OS (ubuntu-latest)","SUCCESS Golden
fixtures cross-OS (windows-latest)","SUCCESS Golden fixtures cross-OS (macos-latest)","SUCCESS
Dependency license scan (no copyleft in runtime deps)","SUCCESS Built wheel smoke (install +
launch)"],"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}

=== #321 checks ===
{"checks":["SUCCESS Ruff lint","SUCCESS Leak scan (PII / attribution guard)","SUCCESS Mypy
(lenient baseline)","SUCCESS Syntax sweep + import smoke (no GPU stack)"," Full test suite
(offline, mocked)","SUCCESS Golden fixtures cross-OS (ubuntu-latest)","SUCCESS Golden fixtures
cross-OS (windows-latest)","SUCCESS Golden fixtures cross-OS (macos-latest)","SUCCESS Dependency
license scan (no copyleft in runtime deps)"," Built wheel smoke (install +
```

launch)"],"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}

103. assistant (2026-06-22T12:34:48.390Z)

Status before I merge:

- **PR [#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319) (code fix + coverage)** ? `mergeStateStatus: CLEAN`, **all 10 CI checks green** (incl. full test suite, ruff, mypy, cross-OS golden fixtures, wheel smoke). Ready to merge.
- **PR [#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321) (doc resolution)** ? green except "Full test suite" + "wheel smoke" still finishing (doc-only; will pass).

I've launched an adversarial verification workflow over the whole change set (code correctness, doc-claim accuracy / no fabricated provenance, test rigor) as the pre-merge gate. I'll merge **#319 first, then #321** once that completes clean and #321's CI finishes. Awaiting the workflow result.

104. user (2026-06-22T12:34:50.195Z)

once the PRs are merged can you identify why the master/origin is broken and try to fix it?

105. assistant (2026-06-22T12:34:56.382Z)

Let me check the verification workflow result and current CI state on both PRs.

106. user (2026-06-22T12:35:00.543Z)

```
=== #319 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","state":"OPEN"}
=== #321 ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE","pending":["Full test suite (offline, mocked)"],"state":"OPEN"}
```

107. assistant (2026-06-22T12:35:13.555Z)

#319 is CLEAN and ready. Let me check whether the verification workflow finished (its result is my pre-merge gate):

108. user (2026-06-22T12:35:15.889Z)

```
agent-a687cc70ecfd80619.jsonl
agent-a687cc70ecfd80619.meta.json
agent-a8fc07a323b7c4b46.jsonl
agent-a8fc07a323b7c4b46.meta.json
agent-a9c6b91377cc917c1.jsonl
agent-a9c6b91377cc917c1.meta.json
journal.jsonl
```

109. user (2026-06-22T12:35:20.943Z)

```
=== journal tail (final result) ===
{"type":"started","key":"v2:6535077d96ce0ec55f80c1bb3601945440739a596e86fbfec8f7bc3a52d5b410","agentId":"a9c6b91377cc917c1"}
{"type":"started","key":"v2:b0538960e0467a5b260ceac6f9e4a4ea11f0bfb0d52c03e1339052e0ef24c2a1","agentId":"a687cc70ecfd80619"}
{"type":"started","key":"v2:6aad32dd292e3cc206a71641533f73921d9386760370e6d35386361515a85412","agentId":"a8fc07a323b7c4b46"}
```

110. assistant (2026-06-22T12:35:39.465Z)

The verification workflow is still running (3 agents in progress). Merge is gated on it. Meanwhile, the master-breakage investigation is read-only and independent ? let me start

understanding what's actually broken on master while the workflow finishes. Let me check master's CI and reproduce the breakage.

111. user (2026-06-22T12:35:44.710Z)

```
=== recent CI runs on master ===
success CI      48f9d20c
cancelled       CI      e221cb92
success CI      d301a7fd
success CI      fa2342c0
success CI      7a40434a
success CI      5c2eec81

=== latest workflow run for master HEAD 48f9d20 ===
[{"conclusion":"success","databaseId":27952730607,"headSha":"48f9d20c29e0a5676fd4c03892e9d7d44ad
428b7","workflowName":"CI"},{"conclusion":"cancelled","databaseId":27952716794,"headSha":"e221cb
928ca1595102fbf7ad1bc0e64727bacb35","workflowName":"CI"},{"conclusion":"success","databaseId":27
949441305,"headSha":"d301a7fdb273a6237f39f9cc5e5d436b1c3271cb","workflowName":"CI"}]
```

112. assistant (2026-06-22T12:36:24.072Z)

Master's CI is actually **green** (commit 48f9d20 CI = success). So "broken" is the **local** full-suite failure I already hit: `test_served_gate_a_regression` (skipped in CI because its corpus manifest is gitignored, but it runs locally). Let me characterize it precisely now ? read the full test and its history ? so I'm ready to fix it properly after the merge.

113. user (2026-06-22T12:36:24.662Z)

```
1       """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3       The flagged gap (#146): the served default flip (``default_segmenter=fusion_hybrid`` +
4       ``min_crossings=2``) is **mock-tested only** on real video (``test_fusion_hybrid.py``
drives a
5       MagicMock runner). The offline ablation litmus
(``test_ablation.test_litmus_reproduces_gate_a``)
6       validates the Gate-A *signal* ? but on the **permissive proposal windowing**
7       (``distill_local.PROPOSAL``), NOT the production operating point. This module closes the
gap: it
8       runs the real ``FusionSegmenter`` decision seams (``_enrich_candidate_stats`` net-
crossings +
9       ``_select_for_handoff`` Gate A) over windows from the **production**
10      ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
11      the served path's measured behaviour.
12
13      GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14      CI where the golden trajectories are absent; runs on the owner box where they are
present, over
15      **whatever golden clips are currently on disk** (``_golden_specs`` requires >= 6).
16
17      CORPUS NOTE ? re-grounded 2026-06-18 (the golden set is mutable and grows over time).
The
18      original 6 *single-court* golden clips this litmus was first calibrated on (2026-06-14)
have been
19      retired from the box ? their labels are gone ? so the present golden set is the 9
**multicourt**
20      clips now in ``output/`` (mahadevpura_1/2, GX0101xx, GX0x0094, Badminton_BXH_2,
Boxhill_Doubles).
21      On that corpus the served path is **recall-starved**: WASB tracks a single shuttle while
several
22      courts rally at once, so ``trajectory_to_action_windows`` emits FEWER windows than there
are
```

```
    rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A only ever
DROPS
    windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
    not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL windowing
the same
    gate is ~F1-neutral (?F1 ? -0.001) while still cutting over-segmentation (1.16?0.64).

    The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
    and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
    (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
    **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
    litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
    re-measurement as the corpus grows.
    """
    from __future__ import annotations

    import json
    import os

    import pytest

    # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
    # (CI), skip the whole module rather than erroring at import.
    pytest.importorskip("cv2")

    from backend.eval import served_gate_a as sga  # noqa: E402
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  # noqa: E402
    from backend.pipeline.detectors import rally_gate  # noqa: E402
    from backend.eval.calibrate_wasb import load_golden_gts  # noqa: E402
    from backend.eval.metrics import score  # noqa: E402
    from backend.eval import segmentation_metrics as _segm  # noqa: E402

    # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
    # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
    # out-of-tree checkout.
    _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    MANIFEST = os.environ.get(
        "SERVED_GATE_A_MANIFEST",
        os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
    )


    def _golden_specs():
        """Return the manifest specs whose trajectory + label files are all present, else
[]."""
        if not os.path.isfile(MANIFEST):
            return []
        with open(MANIFEST, encoding="utf-8") as f:
            specs = json.load(f)
        ok = [s for s in specs
              if os.path.isfile(str(s.get("trajectory", "")))
              and os.path.isfile(str(s.get("labels", "")))]
        return ok if len(ok) >= 6 else []


    _SPECS = _golden_specs()
```

```
76      _skip = pytest.mark.skipif(
77          not _SPECS,
78          reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
79      )
80
81
82      # ------------------------------------------------------------------ served-path
litmus
83
84      @_skip
85      def test_served_path_runs_on_all_golden_videos():
86          """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
EVERY
87          present golden trajectory and yields a handoff window list for each, for both gate
settings.
88          Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden
set grows."""
89          results = sga.evaluate_manifest(_SPECS)
90          assert len(results) == len(_SPECS)
91          assert len(results) >= 6
92          for r in results:
93              assert r.num_gt > 0
94              assert r.n_handoff_off >= r.n_handoff_on   # Gate A only ever drops windows,
never adds
95
96
97      @_skip
98      def test_served_gate_a_does_not_materially_regress_f1():
99          """SERVED no-material-regression guard: on the production windowing Gate A must not
MATERIALLY
100         hurt F1. On the present recall-starved multicourt corpus the served path is under-
segmented
101         (over-seg ratio < 1), so Gate A ? which only DROPS windows ? trims a few borderline-
true
102         windows and is mildly negative, NOT the ~neutral it was on the original single-court
6. We pin
103         a band that admits this measured mild loss but trips on a material regression."""
104         results = sga.evaluate_manifest(_SPECS)
105         agg = sga.aggregate(results)
106         # Honest measured value (2026-06-18, present 9-clip multicourt golden set): mean
served
107         # ?F1 ? -0.034 (was ? -0.008 on the retired original-6). Band re-measured to admit
the mild
108         # recall-starved loss while still tripping if Gate A materially destroys served F1.
109         assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
110         # And Gate A must never collapse served F1 to ~half the OFF baseline (measured ratio
? 0.83).
111         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
112
113
114     @_skip
115     def test_served_gate_a_does_not_increase_over_segmentation():
116         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
117         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
118         results = sga.evaluate_manifest(_SPECS)
119         agg = sga.aggregate(results)
120         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
121         # Per-video too: the gate is monotone (kept windows ? all windows).
122         for r in results:
123             assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
124
125
126     @_skip
127     def test_served_uses_production_windowing_operating_point():
```

```
        """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
        production windowing, this litmus's premise (and the divergence it documents) is
revisited
        deliberately, not silently."""
        assert sga.PROD_VELOCITY_THRESH == 0.02
        assert sga.PROD_MERGE_GAP == 2.0
        assert sga.PROD_MIN_WINDOW == 2.0


    # ----------------------------------------- Gate A under the permissive PROPOSAL
windowing

    # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
    # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
    # small windows). On the original-6 single-court corpus the Gate-A lift here was +0.074
(6/6,
    # p=0.031; archived). On the present multicourt corpus that lift is GONE ? the gate is
~F1-neutral
    # but still cuts over-segmentation. We pin the present-corpus behaviour, not the retired
+0.074.
    _PROPOSAL = (0.005, 1.0, 0.5)


    @_skip
    def test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg():
        """CONTRAST pin: under the OFFLINE harness's permissive proposal windowing, the SAME
        ``rally_gate`` net-crossing gate the served seam uses behaves DIFFERENTLY than under
the
        production windowing. The durable invariant (any corpus): the gate reduces over-
segmentation
        (seg_on < seg_off), since it only drops windows. On the present multicourt corpus it
is
        ~F1-neutral (?F1 ? -0.001, was +0.074 on the now-retired original-6) ? proving the
served
        behaviour is a property of the operating point + corpus, not a broken gate."""
        d_on, d_off, seg_on, seg_off = [], [], [], []
        for s in _SPECS:
            fps, fw = float(s["fps"]), float(s["frame_width"])
            pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
            wins = TrackNetRunner.trajectory_to_action_windows(
                pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
                merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
            iv = [(w["start"], w["end"]) for w in wins]
            est = rally_gate.estimate_play_axis(pts)
            gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
            on = [(g.start, g.end) for g in gated if g.crossings >= 2]
            gts = load_golden_gts(s["labels"])
            d_on.append(score(on, gts).f1)
            d_off.append(score(iv, gts).f1)
            seg_on.append(_segm.segment_count_ratio(on, gts))
            seg_off.append(_segm.segment_count_ratio(iv, gts))
        n = len(d_on)
        mean_delta = (sum(d_on) - sum(d_off)) / n
        mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n
        # Durable direction-of-effect (corpus-independent): the gate only drops windows, so
under the
        # over-segmenting proposal windowing it lowers the over-seg ratio.
        assert mean_seg_on < mean_seg_off, (mean_seg_on, mean_seg_off)
        # Honest measured snapshot (2026-06-18, present 9-clip multicourt set): mean ?F1 ?
-0.001,
        # over-seg 1.16 ? 0.64 (was +0.074 / 1.68 ? 1.01 on the retired original-6).
~F1-neutral band:
```

```
178         assert mean_delta == pytest.approx(0.0, abs=0.03), mean_delta
179         assert mean_seg_off == pytest.approx(1.16, abs=0.10), mean_seg_off
180         assert mean_seg_on == pytest.approx(0.64, abs=0.10), mean_seg_on
181
```

114. assistant (2026-06-22T12:37:15.945Z)

Now I fully understand the breakage. The test (`test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg`) pins **date-stamped empirical snapshots** (over-seg `mean_seg_off ? 1.16`) measured 2026-06-18 on the local golden corpus. The test's own docstring says these "will need re-measurement as the corpus grows." It got `1.36` because the **golden-corpus-hygiene work (#316) changed the local corpus** ? exactly the kind of shift this litmus is designed to trip on. CI skips it (manifest gitignored), so master's CI is green; only the local run breaks.

This is eval-sensitive (owner's golden corpus), so I must diagnose *why* the number moved before re-pinning. Let me check the verification workflow status and inspect the manifest/corpus state.

115. user (2026-06-22T12:37:22.328Z)


```
=== verification workflow done? (journal tail) ===
{"type":"started","key":"v2:6535077d96ce0ec55f80c1bb3601945440739a596e86fbfec8f7bc3a52d5b410","a
gentId":"a9c6b91377cc917c1"}
{"type":"started","key":"v2:b0538960e0467a5b260ceac6f9e4a4ea11f0bfb0d52c03e1339052e0ef24c2a1","a
gentId":"a687cc70ecfd80619"}
{"type":"started","key":"v2:6aad32dd292e3cc206a71641533f73921d9386760370e6d35386361515a85412","a
gentId":"a8fc07a323b7c4b46"}
{"type":"result","key":"v2:6aad32dd292e3cc206a71641533f73921d9386760370e6d35386361515a85412","ag
entId":"a8fc07a323b7c4b46","result":{"dimension":"doc-vs-code factual accuracy (PR #321 doc edit
vs PR #319 code/tests)","ok":true,"blocking_issues":[],"nits":["Prose 'strips both the
.rallies.csv/.csv suffix' maps to two chained .replace() calls in code (.rallies.csv then .csv),
not a single suffix strip. Accurate in effect and test-covered (v1.csv->v1,
2026-06-22_GX020094.csv->GX020094), but a reader might expect one operation. Optional clarity
only.","The §7b#1 prose links the helper to 'backend/worker/__main__.py' without a line number,
whereas the OLD text it replaced cited :384/:378/:387. Not a fabrication (the file/helper
exist), just slightly less precise than the prior revision."],"evidence":"Every factual claim
verified against origin/fix/worker-train-dated-label-join. (1) Helper exists exactly as named:
backend/worker/__main__.py:274 `def _label_clip_name(fname: str) -> str`. (2) Strips
suffix+prefix: line 288 `name = fname.replace(\".rallies.csv\",\"\").replace(\".csv\",\"\")`;
line 289 `return _LABEL_DATE_PREFIX.sub(\"\", name)`; line 271 `_LABEL_DATE_PREFIX =
re.compile(r\"^\\\\d{4}-\\\\d{2}-\\\\d{2}_\")` ? matches the doc's
`^\\\\d{4}-\\\\d{2}-\\\\d{2}_` literal exactly. (3) Dated + bare both -> bare stem: docstring
examples + passing param test. (4) All three cited tests exist AND pass:
tests/test_worker.py:202 test_label_clip_name_strips_date_prefix (6 params, incl. dated
.rallies.csv, bare .rallies.csv, _proxy kept, dated .csv -> all PASSED); :207
test_cmd_train_detector_joins_dated_labels_to_bare_videos (PASSED, asserts
{GX020094,mahadevpura_1}); :254 test_cmd_train_detector_skips_label_with_no_matching_video
(PASSED, orphan absent). Full suite: `python -m pytest tests/test_worker.py -q` => 29 passed.
(5) PR mapping correct: `gh pr view 319` => headRefName fix/worker-train-dated-label-join; #321
is docs/data-gcs-train-blocker-resolved. §8 tracker row at DATA_IN_GCS.md:239 (under '## 8.
Progress tracker') matches the §7b#1 body and cites #319 + `_label_clip_name strips the ISO-date
prefix`. (6) synth+eval unaffected: synth `else:` branch at line 431 never reads video_by_stem
(only set/used inside `if args.trajectory_source == 'detector'` at 393/411); eval is a separate
local-FS-only path (blocker #2) not in this worker. (7) History recounting matches
origin/master:DATA_IN_GCS.md:206-213 verbatim in substance (fname.replace,
video_by_stem.get(name) vs bare stems, every clip skipped -> <2 videos abort)."}}
{"type":"result","key":"v2:b0538960e0467a5b260ceac6f9e4a4ea11f0bfb0d52c03e1339052e0ef24c2a1","ag
entId":"a687cc70ecfd80619","result":{"dimension":"Test quality / regression-guarding strength of
PR #319 worker-train tests","ok":true,"blocking_issues":[],"nits":["test_cmd_train_detector_abor
ts_clean_when_fewer_than_two_videos_join PASSES on master too (verified) because master aborts
EVERY clip with rc 2 ? so this test alone does not distinguish fixed-vs-broken. It is not relied
on as the regression guard (the join/skip tests are), and it correctly guards the legitimate
clean-abort path + that subprocess.run is never reached, so this is acceptable. Worth a one-line
comment noting it is not a pre-fix differentiator.","The join test's rc==0 also depends on the
GPU/harness boundary being faithfully stubbed (real _resolve_train_detector + stub runner write
```

a real trajectory; only _probe_video_fps_width and subprocess.run are patched). This is the right seam, but the test would also break if the stub_runner contract changed ? acceptable coupling, just noting it is an integration-flavored test, not a unit test."],"evidence":"Ran the new tests on both branches via git worktrees.\n\nFIX branch (origin/fix/worker-train-dated-label-join @ 56c1415): all 12 selected tests PASS (6 parametrized _label_clip_name + 6 cmd_train_detector).\n\nMASTER branch (48f9d20) with the NEW test file copied in: 9 of 10 regression-relevant tests FAIL, exactly as the guard requires:\n- All 6 test_label_clip_name_strips_date_prefix cases FAIL (master has 0 occurrences of _label_clip_name -> AttributeError). Confirmed: `grep -c _label_clip_name master/backend/worker/__main__.py` = 0; master does `name = fname.replace('.rallies.csv','').replace('.csv','')` (line 384, no date strip).\n- test_cmd_train_detector_joins_dated_labels_to_bare_videos FAILS `assert 2 == 0`. Captured stdout: `[worker] no video for '2026-06-22_GX020094' under .../videos/ ? skipping.` + `[worker] no video for '2026-06-21_mahadevpura_1' ... skipping.` -> `need >=2 videos ... got 0.` This is the exact regression mechanism: master leaves the ISO-date prefix on the derived name, so video_by_stem.get(name) (keyed by bare stem) misses every dated label -> all clips skipped -> rc 2.\n- test_cmd_train_detector_skips_label_with_no_matching_video and test_cmd_train_detector_skips_unprobeable_and_trajless_videos both FAIL with rc 2 on master for the same reason.\n\nSource fix (backend/worker/__main__.py): adds _LABEL_DATE_PREFIX = re.compile(r'^\\\\d{4}-\\\\d{2}-\\\\d{2}_') and _label_clip_name(); cmd_train now calls `name = _label_clip_name(fname)` instead of the naive replace.\n\nMechanism / non-tautology checks:\n- The join test does NOT mock away the code under test: RALLY_DETECTOR_IMPL=stub + real _resolve_train_detector build a genuine StubRunner whose run_predict writes a real trajectory CSV (backend/pipeline/detectors/stub_runner.py run_predict returns out_csv); only _probe_video_fps_width and subprocess.run (GPU/harness boundary) are patched. The date-strip join + manifest assembly are real.\n- On the FIX branch, the unprobeable/trajless test stdout shows the two skip branches fire by the RIGHT (date-stripped) name and via DISTINCT messages: `could not probe fps/width for 'badfps' ... skipping (won't guess).` and `detector produced no trajectory for 'notraj' ... skipping.` -> proves badfps hits the probe-skip branch and notraj hits the no-trajectory branch independently (not collapsed). Manifest asserts exactly {good1, good2}.\n- _FakeRunner (unprobeable test) replaces only trajectory production; _label_clip_name and the video_by_stem join remain real and unpatched.\n- Assertions check concrete manifest spec names and rc, never trivially-true conditions. The abort test additionally guards subprocess.run is never reached (monkeypatched to pytest.fail).\n\nNo assertion is trivially always-true; monkeypatches target only the GPU/probe/harness seam, not the helper or join under test."}}

```
=== manifest present? how many specs resolve? ===
manifest exists: True
total specs: 15
resolving (traj+label present): 15
  OK  traj=1 label=1  adarsh_avi_singles
  OK  traj=1 label=1  kushagra_singles
  OK  traj=1 label=1  largetest_doubles
  OK  traj=1 label=1  gbaaddy
  OK  traj=1 label=1  testlarge_short
  OK  traj=1 label=1  mahadevpura_singles
  OK  traj=1 label=1  mahadevpura_2
  OK  traj=1 label=1  GX010128
  OK  traj=1 label=1  mahadevpura_1
  OK  traj=1 label=1  GX030094
  OK  traj=1 label=1  Badminton_BXH_2
  OK  traj=1 label=1  GX010141
  OK  traj=1 label=1  GX010137
  OK  traj=1 label=1  Boxhill_Doubles
  OK  traj=1 label=1  GX020094
```

116. assistant (2026-06-22T12:38:08.157Z)

Now the breakage is fully diagnosed ? and it's revealing. The manifest now resolves **15 clips, not the 9** the test was calibrated on (2026-06-18). The six newly-present clips are exactly the **original-6 single-court clips** (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`, `testlarge_short`, `mahadevpura_singles`) that the docstring said were "retired from the box ? their labels are gone." The golden-corpus-hygiene migration (#316) **restored the original-6 labels+trajectories** from the F: archive. Single-court clips over-segment (~1.68) vs the recall-starved multicourt ones (~0.64), so the blend moved `mean_seg_off` 1.16 ? 1.36. **The litmus tripped correctly on a real corpus change ? not a code

regression.**

Verification workflow: 2 of 3 agents back, both **ok=true, zero blocking issues** (doc-accuracy confirmed no fabricated claims; test-rigor independently confirmed the tests fail on master pre-fix). Waiting on the 3rd (code-correctness). Let me check its progress and #321 CI.

117. user (2026-06-22T12:38:13.201Z)

```
=== workflow: 3rd agent result present? ===
2
=== #321 full-suite check ===
{"mergeStateStatus":"CLEAN","pending":[]}
```

118. assistant (2026-06-22T12:38:23.019Z)

#321 is now CLEAN too. The 3rd verification agent (code-correctness) is still running. Let me check its progress:

119. user (2026-06-22T12:38:25.362Z)


```
=== code-correctness agent: last activity ===
{"parentUuid":"154c4a81-9fa1-48af-
aba6-24fb13ec533d","isSidechain":true,"agentId":"a9c6b91377cc917c1","message":{"model":"claude-o
pus-4-
8","id":"msg_012g7Gn4ko1Eg5UFeune2fCm","type":"message","role":"assistant","content":[{"type":"t
ool_use","id":"toolu_01GVkmQxhcjBuWrSkQ8Qy3XA","name":"PowerShell","input":{"command":"git
worktree remove C:\\Users\\avidu\\Projects\\_pr319_review --force 2>&1; git worktree prune;
Write-Host \"cleaned\"","description":"Remove temporary review worktree"},"caller":{"type":"dire
ct"}}],"stop_reason":"tool_use","stop_sequence":null,"stop_details":null,"usage":{"input_tokens"
:2,"cache_creation_input_tokens":1107,"cache_read_input_tokens":48993,"output_tokens":236,"serve
r_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_cr
eation":{"ephemeral_1h_input_tokens":0,"ephemeral_5m_input_tokens":1107},"inference_geo":"not_av
ailable","iterations":[{"input_tokens":2,"output_tokens":236,"cache_read_input_tokens":48993,"ca
che_creation_input_tokens":1107,"cache_creation":{"ephemeral_5m_input_tokens":1107,"ephemeral_1h
_input_tokens":0},"type":"message"}],"speed":"standard"},"diagnostics":null},"requestId":"req_01
1CcJJykiMxJX5Yu4gvKZEx","attributionAgent":"workflow-subagent","type":"assistant","uuid":"516ccf
38-4590-4dc0-9cd2-9eafdd8078f7","timestamp":"2026-06-
22T12:37:59.079Z","userType":"external","entrypoint":"claude-
desktop","cwd":"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer","sessionId":"73a3938e-1da3-4e28-bf
```


120. assistant (2026-06-22T12:38:32.711Z)

The code-correctness agent is finalizing (cleaning up its review worktree). Let me get its verdict:

121. user (2026-06-22T12:38:39.603Z)

```
dimension: correctness
ok: True
blocking_issues: []
nits: [
  "Helper strips only the LEADING date, so a pathological label like
`2026-06-22_2026-06-22_foo.rallies.csv` yields `2026-06-22_foo` (inner date kept). This is
actually correct given the contract (it round-trips to a hypothetical
`videos/2026-06-22_foo.mp4`), but the implicit assumption that no clip <name> begins `YYYY-MM-
DD_` is worth a one-line note in the docstring or DATA_IN_GCS naming contract.",
  "Degenerate inputs `2026-06-22_.rallies.csv` and `.rallies.csv` produce an empty `name`. The
contract guarantees a non-empty <name>, and an empty stem merely misses the video join and is
skipped (no crash), so this is theoretical only \u2014 but a defensive guard/skip-with-warning
on empty `name` would make the helper robust to a malformed bucket object.",
  "A structurally-valid-but-impossible date such as `2026-13-99_` is treated as a date prefix
(regex is structural, not calendar-validated). Harmless and not producible under the authoring-
date convention; calendar validation would be overkill here."
```

]
evidence: backend/worker/__main__.py:23 `import re` present at module level.
backend/worker/__main__.py:268-289 helper: `_LABEL_DATE_PREFIX =
re.compile(r"^\d{4}-\d{2}-\d{2}_")` is ^-anchored; `_label_clip_name` does
`fname.replace('.rallies.csv','').replace('.csv','')` then `_LABEL_DATE_PREFIX.sub('', name)`.
cmd_train (lines 407-435): `name = _label_clip_name(fname)` feeds the join
`video_by_stem.get(name)` (411), detector spec `{'name': name,...}` (429), synth traj
`f'{name}_traj.csv'` (433) and synth spec (435); `local_label` fetch correctly uses the full
dated `fname` (409) so cross-date same-clip labels don't collide on disk. Adversarial probe (run
against the actual regex): anchored ? `foo_2026-06-22_bar`->unchanged,
`2026-06-22-foo`->unchanged, `20260622_foo`->unchanged; date-stripped correctly ?
`2026-06-22_GX020094`->`GX020094`, `2026-06-21_mahadevpura_1`->`mahadevpura_1`. Legacy
regression proof: for every no-date label (`GX020094_proxy`, `GX010128`, `mahadevpura-1`,
`Boxhill Doubles_proxy`, `mahadevpura_smoke_180s`) new()==old() byte-for-byte ? change is purely
additive. Naming contract LOCKED (docs/DATA_IN_GCS.md:176-179 `labels/<YYYY-MM-
DD>_<name>.rallies.csv`; :141 `videos/<name>.<ext>`), so the date-prefixed-video edge case is
precluded by convention. Verification on a clean detached worktree at origin/fix/worker-train-
dated-label-join: tests/test_worker.py = 29 passed (incl. parametrized helper cases +
dated->bare join regression + orphan-skip + unprobeable/trajless-skip); run_all_tests.py = 1423
passed, 7 skipped; `ruff check backend/worker/__main__.py tests/test_worker.py` = All checks
passed; `mypy backend/worker/__main__.py` = Success, no issues. Branch contains only its own 3
commits vs origin/master (a2c8be0 fix, b3b4952 test, 56c1415 ruff F401 cleanup); no

122. assistant (2026-06-22T12:39:01.287Z)

Verification workflow complete: `all_ok: true` ? all 3 dimensions (code-correctness, doc-
accuracy, test-rigor) returned ok=true with **zero blocking issues**. The nits are
minor/theoretical (date-prefix edge cases precluded by the locked naming contract; wording). The
test-rigor agent independently confirmed via worktrees that the new tests fail on master pre-fix
and pass on the fix branch.

Both PRs are CLEAN, the user's conditions are met (coverage 82%?84% worker / 85.59% overall;
docs fixed). Merging now ? **#319 first** (so the doc's reference resolves to a merged PR), then
#321.

123. user (2026-06-22T12:39:12.967Z)

```
=== switch off the doc branch to master + update ===
fatal: 'master' is already used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/tier1-windowing'
 * branch            master     -> FETCH_HEAD
Already up to date.

=== merge #319 (code fix) squash + delete branch ===

=== merge #321 (doc) squash + delete branch ===
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'
```

124. assistant (2026-06-22T12:39:23.098Z)

The `--delete-branch` cleanup tripped over `master` being checked out in another worktree
(`tier1-windowing`). The API merge may still have succeeded. Let me check the actual state of
both PRs:

125. user (2026-06-22T12:39:26.593Z)

```
=== #319 state ===
{"mergeCommit":"b72e6de027d6b5f2a35076a4ac259cf7907a62eb","mergedAt":"2026-06-
22T12:39:08Z","state":"MERGED"}
=== #321 state ===
{"mergeStateStatus":"UNKNOWN","mergeable":"UNKNOWN","mergedAt":"2026-06-
22T12:39:12Z","state":"MERGED"}
=== current branch ===
```

```
docs/data-gcs-train-blocker-resolved
```

126. assistant (2026-06-22T12:39:39.049Z)

```
Both PRs are **MERGED** ? (#319 ? b72e6de at 12:39:08Z; #321 at 12:39:12Z). Only the local
`--delete-branch` cleanup failed (master lives in another worktree ? `tier1-windowing`). Let me
confirm origin/master has both, prune remote branches, and set up for the master fix.
```

127. user (2026-06-22T12:39:42.928Z)

```
 - [deleted]         (none)     -> origin/docs/data-gcs-train-blocker-resolved
 - [deleted]         (none)     -> origin/fix/worker-train-dated-label-join
   48f9d20..db74f9b  master     -> origin/master
=== origin/master top 4 (should include #319 + #321) ===
db74f9b docs(data): mark DATA_IN_GCS §7b blocker #1 resolved by #319 (#321)
b72e6de fix(worker): strip ISO-date prefix when joining labels to videos in train (#319)
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
=== remote feature branches still around? ===
(both remote branches deleted)
```

128. assistant (2026-06-22T12:40:50.205Z)

```
Both merges are clean on origin/master, remote branches pruned.

Now the master fix. Let me mark the new phase and set up a branch off the freshly-merged master.
The fix requires **re-measuring** the eval snapshots on the current 15-clip corpus (this is
eval-sensitive owner data, so I measure before changing anything).
```

129. user (2026-06-22T12:40:51.075Z)

```
Chapter marked: "Fix broken master (eval litmus)". Continue your current work.
```

130. user (2026-06-22T12:40:58.087Z)

```
=== clean tree? ===
(clean tracked)
=== create fix branch off origin/master (has #319+#321) ===
branch 'fix/served-gate-a-regrounded-15clip' set up to track 'origin/master'.
Switched to a new branch 'fix/served-gate-a-regrounded-15clip'
=== HEAD ===
db74f9b docs(data): mark DATA_IN_GCS §7b blocker #1 resolved by #319 (#321)
=== scratch gitignored? ===
scratch/x
scratch ignored
```

131. assistant (2026-06-22T12:41:04.176Z)

```
Now let me write a measurement script that reproduces every pinned quantity in the litmus on the
current 15-clip corpus ? so I re-ground from real measurement, not guesswork:
```

132. assistant (2026-06-22T12:41:26.117Z)

```
"""Re-measure every snapshot pinned by tests/test_served_gate_a_regression.py on the CURRENT
local golden corpus (now 15 clips after the #316 GCS migration restored the original-6 labels).

Prints: the resolved clip set, the proposal-windowing quantities (mean_delta, mean_seg_off,
mean_seg_on) with a per-clip breakdown, and the served-path aggregate (mean_delta_f1, f1_on/off,
seg_ratio_on/off). Also checks the DURABLE invariants the test relies on so we can confirm the
move is a corpus change, not a code regression. GPU-free / Gemini-free.
"""
import json
import os
```

```
from backend.eval import served_gate_a as sga
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.detectors import rally_gate
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.metrics import score
from backend.eval import segmentation_metrics as _segm

_REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MANIFEST = os.environ.get(
    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
)
_PROPOSAL = (0.005, 1.0, 0.5)


def golden_specs():
    with open(MANIFEST, encoding="utf-8") as f:
        specs = json.load(f)
    return [s for s in specs
            if os.path.isfile(str(s.get("trajectory", "")))
            and os.path.isfile(str(s.get("labels", "")))]


def main():
    specs = golden_specs()
    print(f"MANIFEST: {MANIFEST}")
    print(f"resolved specs: {len(specs)}\n")

    # ---- proposal-windowing contrast test quantities ----
    rows = []
    d_on, d_off, seg_on, seg_off = [], [], [], []
    for s in specs:
        fps, fw = float(s["fps"]), float(s["frame_width"])
        pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
        wins = TrackNetRunner.trajectory_to_action_windows(
            pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
            merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
        iv = [(w["start"], w["end"]) for w in wins]
        est = rally_gate.estimate_play_axis(pts)
        gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
        on = [(g.start, g.end) for g in gated if g.crossings >= 2]
        gts = load_golden_gts(s["labels"])
        f1_on, f1_off = score(on, gts).f1, score(iv, gts).f1
        sr_on, sr_off = _segm.segment_count_ratio(on, gts), _segm.segment_count_ratio(iv, gts)
        d_on.append(f1_on); d_off.append(f1_off); seg_on.append(sr_on); seg_off.append(sr_off)
        rows.append((s.get("name", "?"), len(gts), len(iv), len(on), sr_off, sr_on, f1_off,
f1_on))

    n = len(d_on)
    mean_delta = (sum(d_on) - sum(d_off)) / n
    mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n

    print("PROPOSAL-WINDOWING per clip (name, n_gt, n_off, n_on, seg_off, seg_on, f1_off,
f1_on):")
    for r in sorted(rows, key=lambda x: -x[4]):
        print(f"  {r[0]:<22} gt={r[1]:<3} off={r[2]:<3} on={r[3]:<3} "
              f"seg_off={r[4]:.2f} seg_on={r[5]:.2f} f1_off={r[6]:.3f} f1_on={r[7]:.3f}")
    print()
    print(f"[proposal] n={n}")
    print(f"[proposal] mean_delta (f1_on - f1_off) = {mean_delta:+.4f}   (was pinned ~0.0
±0.03)")
    print(f"[proposal] mean_seg_off               = {mean_seg_off:.4f}   (was pinned 1.16
±0.10)")
    print(f"[proposal] mean_seg_on                = {mean_seg_on:.4f}   (was pinned 0.64
±0.10)")
```

```
    print(f"[proposal] DURABLE invariant mean_seg_on < mean_seg_off : "
          f"{mean_seg_on < mean_seg_off}  ({mean_seg_on:.3f} < {mean_seg_off:.3f})")
    print()

    # ---- served-path test quantities ----
    results = sga.evaluate_manifest(specs)
    agg = sga.aggregate(results)
    print("SERVED-PATH aggregate:")
    print(f"[served] mean_delta_f1   = {agg['mean_delta_f1']:+.4f}   (band pinned [-0.06,
0.03])")
    print(f"[served] mean_f1_on      = {agg['mean_f1_on']:.4f}")
    print(f"[served] mean_f1_off     = {agg['mean_f1_off']:.4f}")
    print(f"[served] f1_on/f1_off    = {agg['mean_f1_on'] / max(agg['mean_f1_off'], 1e-9):.3f} "
          f"(must be >= 0.5)")
    print(f"[served] mean_seg_on     = {agg['mean_seg_ratio_on']:.4f}")
    print(f"[served] mean_seg_off    = {agg['mean_seg_ratio_off']:.4f}")
    print(f"[served] DURABLE seg_on <= seg_off : "
          f"{agg['mean_seg_ratio_on'] <= agg['mean_seg_ratio_off'] + 1e-9}")
    # per-video monotonicity (Gate A only drops windows)
    mono = all(r.seg_ratio_on <= r.seg_ratio_off + 1e-9 for r in results)
    drops = all(r.n_handoff_off >= r.n_handoff_on for r in results)
    print(f"[served] per-video seg monotone (on<=off): {mono}")
    print(f"[served] per-video Gate A only drops (off>=on handoffs): {drops}")
    print(f"[served] num videos: {len(results)}  (all n_gt>0: {all(r.num_gt > 0 for r in
results)})")


if __name__ == "__main__":
    main()
```

133. user (2026-06-22T12:41:26.137Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\measure_served_gate_a_15clip.py (file state is current in your context ? no need
to Read it back)
```

134. user (2026-06-22T12:41:29.388Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\measure_served_gate_a_15clip.py", line 12, in <module>
    from backend.eval import served_gate_a as sga
ModuleNotFoundError: No module named 'backend'
```

135. user (2026-06-22T12:41:44.300Z)

```
MANIFEST: C:\Users\avidu\Projects\badminton-highlight-indexer\output\human_lovo_manifest.json
resolved specs: 15

PROPOSAL-WINDOWING per clip (name, n_gt, n_off, n_on, seg_off, seg_on, f1_off, f1_on):
  GX020094               gt=40  off=99  on=28  seg_off=2.48 seg_on=0.70 f1_off=0.432 f1_on=0.559
  testlarge_short        gt=6   off=14  on=10  seg_off=2.33 seg_on=1.67 f1_off=0.300 f1_on=0.375
  GX030094               gt=41  off=79  on=42  seg_off=1.93 seg_on=1.02 f1_off=0.350 f1_on=0.361
  mahadevpura_singles    gt=31  off=57  on=39  seg_off=1.84 seg_on=1.26 f1_off=0.455 f1_on=0.571
  adarsh_avi_singles     gt=96  off=172 on=84  seg_off=1.79 seg_on=0.88 f1_off=0.597 f1_on=0.722
  largetest_doubles      gt=49  off=86  on=38  seg_off=1.76 seg_on=0.78 f1_off=0.593 f1_on=0.667
  gbaaddy                gt=48  off=82  on=47  seg_off=1.71 seg_on=0.98 f1_off=0.231 f1_on=0.274
  GX010137               gt=34  off=56  on=25  seg_off=1.65 seg_on=0.74 f1_off=0.689 f1_on=0.746
  Boxhill_Doubles        gt=43  off=59  on=50  seg_off=1.37 seg_on=1.16 f1_off=0.235 f1_on=0.258
  GX010128               gt=52  off=51  on=43  seg_off=0.98 seg_on=0.83 f1_off=0.291 f1_on=0.316
  GX010141               gt=29  off=26  on=13  seg_off=0.90 seg_on=0.45 f1_off=0.400 f1_on=0.143
  Badminton_BXH_2        gt=44  off=29  on=24  seg_off=0.66 seg_on=0.55 f1_off=0.027 f1_on=0.029
  kushagra_singles       gt=32  off=20  on=17  seg_off=0.62 seg_on=0.53 f1_off=0.154 f1_on=0.163
  mahadevpura_2          gt=40  off=14  on=14  seg_off=0.35 seg_on=0.35 f1_off=0.074 f1_on=0.074
```

```
  mahadevpura_1          gt=10  off=1   on=0   seg_off=0.10 seg_on=0.00 f1_off=0.000 f1_on=0.000

[proposal] n=15
[proposal] mean_delta (f1_on - f1_off) = +0.0287   (was pinned ~0.0 ±0.03)
[proposal] mean_seg_off               = 1.3640   (was pinned 1.16 ±0.10)
[proposal] mean_seg_on                = 0.7919   (was pinned 0.64 ±0.10)
[proposal] DURABLE invariant mean_seg_on < mean_seg_off : True  (0.792 < 1.364)

SERVED-PATH aggregate:
[served] mean_delta_f1   = -0.0233   (band pinned [-0.06, 0.03])
[served] mean_f1_on      = 0.2134
[served] mean_f1_off     = 0.2366
[served] f1_on/f1_off    = 0.902 (must be >= 0.5)
[served] mean_seg_on     = 0.4696
[served] mean_seg_off    = 0.5825
[served] DURABLE seg_on <= seg_off : True
[served] per-video seg monotone (on<=off): True
[served] per-video Gate A only drops (off>=on handoffs): True
[served] num videos: 15  (all n_gt>0: True)
```

136. assistant (2026-06-22T12:43:43.521Z)

Now I have the complete re-measured picture. Critically:

- **All durable invariants hold** (`mean_seg_on < mean_seg_off`, Gate A only drops windows, per-video monotone) ? this is a **corpus change, not a code regression**.
- Only the **proposal-windowing test's absolute snapshots** broke: `mean_seg_off` 1.16?**1.36**, `mean_seg_on` 0.64?**0.79**. The served-path tests still pass (they use ranges).
- A real finding: with the original-6 single-court clips back, the proposal-windowing gate is now mildly **F1-positive** (?F1 +0.029, was ?-0.001 on 9 clips) ? the archived +0.074 story partially re-entering the blend.

The fix = honest re-grounding to the 15-clip corpus (the test's docstring explicitly says snapshots "will need re-measurement as the corpus grows"). Let me update the snapshots **and** the now-stale narrative/comments (honesty matters more than minimalism ? a passing-but-lying test is worse).

137. user (2026-06-22T12:43:54.692Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)

138. user (2026-06-22T12:44:09.217Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)

139. assistant (2026-06-22T12:44:13.639Z)

Now the proposal-windowing test (docstring + the failing snapshot assertions):

140. user (2026-06-22T12:44:19.364Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)

141. user (2026-06-22T12:44:28.225Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)

142. assistant (2026-06-22T12:44:40.179Z)

Now run the full litmus module to confirm all 5 tests pass on the 15-clip corpus:

143. user (2026-06-22T12:45:02.201Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 5 items

tests/test_served_gate_a_regression.py::test_served_path_runs_on_all_golden_videos PASSED [ 20%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_materially_regress_f1 PASSED
[ 40%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_increase_over_segmentation
PASSED [ 60%]
tests/test_served_gate_a_regression.py::test_served_uses_production_windowing_operating_point
PASSED [ 80%]
tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_ov
er_seg PASSED [100%]

============================== 5 passed in 20.02s ==============================
```

144. assistant (2026-06-22T12:45:08.143Z)

All 5 litmus tests pass. Let me run the full suite to confirm master is unbroken locally, and check for any stray references to the old 9-clip narrative elsewhere:

145. user (2026-06-22T12:45:09.056Z)

```
Found 2 files
tests\test_served_gate_a_regression.py
docs\QUALITY_ITERATIONS.md
```

146. assistant (2026-06-22T12:45:18.587Z)

Let me run the full suite (confirming master is unbroken) in the background while I check what `QUALITY_ITERATIONS.md` says about the 9-clip set:

147. user (2026-06-22T12:45:19.968Z)

```
Command running in background with ID: bqku4e6i9. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bqku4e6i9.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

148. user (2026-06-22T12:45:23.724Z)

```
588-
589-> **Re-grounded 2026-06-18.** The original-6 single-court golden clips this litmus was
calibrated on have been
590:> retired from the box (their labels are gone); the present golden set is **9 multicourt
clips**. On that
591-> recall-starved corpus (WASB tracks one shuttle while several courts rally ? served over-
seg ratio < 1) the
592-> served Gate-A ?F1 is now **?0.034** (was ?0.008), and under the permissive proposal
windowing the gate is
593:> **~F1-neutral** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer
reproduces the +0.074
594-> live. The canonical **+0.074 / 6-of-6 / p=0.031** win remains a real result of the
```

```
original-6 corpus,
595-> preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-
experiment/README.md).
```

149. assistant (2026-06-22T12:45:32.236Z)

`docs/QUALITY_ITERATIONS.md` carries the **same now-stale 9-clip narrative** (the ablation backbone doc, which the owner keeps as "honest measured results only"). Leaving it stale would contradict the re-grounded test and be dishonest. Let me read its context and update it to match the 15-clip reality:

150. user (2026-06-22T12:45:32.966Z)

```
576     - `backend/config/models.py` ? `FusionConfig.min_crossings` default **2 ? 0** (Gate A
**OFF** by default).
577     - Tests (`tests/test_config.py`, `tests/test_fusion_hybrid.py`) restored to the Gate-A-
OFF defaults; the
578       **persisted-superset invariant is preserved** (gating only ever filtered the AI
handoff, never persistence).
579     - `eval_baselines/experiment_leaderboard.json` ? the #146 `promotion` record carries a
`reverted (eval?serve)`
580       note.
581
582     **Gate A is NOT deleted ? it remains opt-in** (`indexing.fusion.min_crossings: 2`) for
footage with more
583     held-shuttle/service-feed false positives than the golden 6, and the offline ablation
evidence below stands
584     (it is a real proposal-windowing win). The path back to serving it is to **align the
served windowing toward
585     the proposal operating point** and re-run the #148 litmus to confirm the lift transfers.
The #148 litmus is
586     now the standing guard: it pins the honest measured behaviour on the **present golden
corpus** and trips if a
587     future windowing/corpus change silently shifts it.
588
589     > **Re-grounded 2026-06-18.** The original-6 single-court golden clips this litmus was
calibrated on have been
590     > retired from the box (their labels are gone); the present golden set is **9 multicourt
clips**. On that
591     > recall-starved corpus (WASB tracks one shuttle while several courts rally ? served
over-seg ratio < 1) the
592     > served Gate-A ?F1 is now **?0.034** (was ?0.008), and under the permissive proposal
windowing the gate is
593     > **~F1-neutral** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer
reproduces the +0.074
594     > live. The canonical **+0.074 / 6-of-6 / p=0.031** win remains a real result of the
original-6 corpus,
595     > preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-
experiment/README.md).
596
597     ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*
598     **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The
measured win
599     (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the
600     [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
clears the honest
601     bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the
eval-only Gate A
```

151. user (2026-06-22T12:45:55.388Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md has been
```

updated successfully. (file state is current in your context ? no need to Read it back)

152. assistant (2026-06-22T12:46:02.906Z)

Now let me lint the changed test file while the full suite finishes in the background:

153. user (2026-06-22T12:46:05.912Z)

```
=== ruff on changed test file ===
All checks passed!
=== full suite background status ===
```

154. assistant (2026-06-22T12:46:32.153Z)

Ruff clean. The full suite is still running in the background ? let me wait for it to confirm the previously-failing test is now green and nothing else regressed before I commit. Let me check its progress:

155. user (2026-06-22T12:47:51.284Z)

```
TOTAL                                                10001   1441    86%
Required test coverage of 70% reached. Total coverage: 85.59%
1430 passed, 7 warnings in 148.51s (0:02:28)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

156. assistant (2026-06-22T12:48:02.318Z)

RESULT: PASS ? all tests green. 1430 passed (previously 1429 passed + 1 failed), coverage 85.59%. The litmus is re-grounded and master is unbroken locally.

Let me commit the fix (test + doc), verifying branch hygiene:

157. user (2026-06-22T12:48:12.502Z)

```
warning: in the working copy of 'docs/QUALITY_ITERATIONS.md', LF will be replaced by CRLF the
next time Git touches it
=== staged (only these 2) ===
docs/QUALITY_ITERATIONS.md
tests/test_served_gate_a_regression.py
=== branch + commits ahead of origin/master ===
fix/served-gate-a-regrounded-15clip
[fix/served-gate-a-regrounded-15clip 5edd9f8] test(eval): re-ground served Gate-A litmus to the
15-clip golden corpus
 2 files changed, 50 insertions(+), 40 deletions(-)
=== committed ===
5edd9f8 test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus
```

158. user (2026-06-22T12:48:25.978Z)

```
remote:
branch 'fix/served-gate-a-regrounded-15clip' set up to track 'origin/fix/served-gate-a-
regrounded-15clip'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/served-gate-a-regrounded-15clip -> fix/served-gate-a-regrounded-15clip
```

159. user (2026-06-22T12:48:47.713Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/324

160. assistant (2026-06-22T12:49:09.890Z)

PR **[#324](https://github.com/Khelsutra/badminton-highlight-indexer/pull/324)** opened ?
deliberately **not auto-merged** (it re-grounds eval-quality snapshots and CI can't verify it;
left for owner review per the launch-report/honest-results conventions).

Let me record this corpus-state change as a memory (it affects eval beyond this one test) and
update the live current-state checkpoint.

161. assistant (2026-06-22T12:49:20.982Z)

```
---
name: golden-corpus-grew-9-to-15
description: The local golden eval corpus grew 9?15 clips (original-6 single-court restored by
#316); re-grounds eval snapshots
metadata:
  type: project
---
```

2026-06-22: the local golden eval corpus went from **9 multicourt clips ? 15 clips**. The
GCS
corpus migration ([[gcs-data-source-of-truth]], #316) RESTORED the **original-6 single-court**
labels +
trajectories to the box (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`,
`gbaaddy`,
`testlarge_short`, `mahadevpura_singles`), so `output/human_lovo_manifest.json` now resolves all
15
(9 multicourt + 6 single-court). Earlier eval docs/tests that say the original-6 are "retired /
labels
gone / present set is 9 multicourt" are now STALE.

Why it matters: any litmus/ablation pinned on the 9-clip snapshot trips on this change.
Concretely,
`tests/test_served_gate_a_regression.py` failed locally (`mean_seg_off` 1.16?**1.36**,
`mean_seg_on`
0.64?**0.79**) ? it SKIPS in CI (manifest gitignored), so master CI stayed green while the local
run
broke. Re-grounded in **PR #324** (test docstring + `docs/QUALITY_ITERATIONS.md`); durable
invariants
all held ? corpus change, not a code regression. **Notable:** with the original-6 (the archived
+0.074 Gate-A win) back in the blend, the proposal-windowing gate is mildly **F1-positive**
again
(?F1 ? +0.029, was ? ?0.001 on 9 clips); served stays mildly negative (? ?0.023).

How to apply: when an eval snapshot trips, first check whether the corpus changed (count
resolved
specs in the manifest) before suspecting a code regression; re-measure on the live set and re-
ground
the snapshot honestly (per [[launch-report-convention]] / [[decision-rigor-norm]]). The 9?15
growth
likely affects OTHER eval narratives too ? worth a wider sweep. Pairs with [[eval-dual-set-
regression]].

162. user (2026-06-22T12:49:21.010Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-
highlight-indexer\memory\golden-corpus-grew-9-to-15.md (file state is current in your context ?
no need to Read it back)

163. user (2026-06-22T12:49:25.192Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
```

```
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      > **Scope of this file = LIVE only:** the current handoff + *today's* checkpoints + open
PRs/decisions. Keep it read-first-short (?1 page). Older checkpoints are archived in [[current-
state-archive]] ? when today's checkpoints age out (next clean-slate day), move them there,
don't let them pile up here.
13
14      **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? Cloud Run scheduler path FIXED + cloud-
validated, [#320] MERGED**). master `48f9d20`. Owner asked Option A (make
`register_scheduler.sh` actually work) + a trigger command + email setup. **Two bugs caught by
an in-cloud dry-run validation** (build the launcher image ? temp dry-run Cloud Run job ? exec):
(1) the launcher container is `COPY . /app` but Cloud Build excludes `.git` ? `launch.sh`'s `git
archive` failed ? **fixed**: `launch.sh` now tars the baked tree when no `.git` + takes the SHA
from `RALLY_GIT_SHA` (+ a `RALLY_DRY_RUN` smoke mode); (2) `gcloud builds submit --tag` has **no
`-f` flag** ? added `cloudbuild.launcher.yaml` + `--config`. Also: `register_scheduler.sh` auto-
grants the SA IAM (compute.instanceAdmin.v1+iam.serviceAccountUser+run.invoker) + bakes the SHA
+ `--run-now`; new **`setup_email.sh`** (Gmail app-pw via STDIN ? Secret Manager + SA
secretAccessor). **VALIDATED in-cloud**: image built, the container ran launch.sh (no .git ?
tar, SHA=4051ed5), staged the tarball, hit "Skipping VM create" (dry-run) ? with the earlier
real-L4 run, the FULL Cloud Scheduler?Cloud Run job?launch.sh?ephemeral-GPU-VM chain is proven.
Temp job+image deleted; nightly bucket kept; GCP $0. **? Owner go-live (all from a LOCAL master
checkout, NOT the droplet): `create_nightly_bucket.sh` (done) ? upload clips/labels to
`gs://?/nightly-inputs/` ? `setup_email.sh` ? `register_scheduler.sh --run-now`. Start on
demand: `gcloud run jobs execute nightly-regression-launcher --region asia-south1`.** [[gcp-vm-
always-delete]] [[working-agreement-merging]]
15
16      **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? FAITHFUL WASB run VALIDATED on a real GCP L4
GPU; follow-up [#318]**). Owner: "use GCP GPU VMs (auth/tooling available)." Spun up an **L4**
(`rally-nightly-val`, asia-south1-b) via `create_gpu_vm.sh`, set up the real WASB-native conda
env + weights (`gs://khelsutra-rally-corpus/models/wasb_badminton_best.pth.tar`), ran
`nightly_reelcount.py` with `wasb_hybrid`/`detector_impl=native`/`skip_ai_handoff=true` on
`gs://?/videos/mahadevpura_smoke_180s.mp4`: **22 reels, DETERMINISTIC across 3 runs, clean PASS
(exit 0), faked regression caught (exit 1, `reel_count 25?22`), cost from real uptime
($0.069/?5.77, 352s @ $0.71/hr)**. **VM DELETED** ($0 idle, no orphan disks ? note `teardown.sh`
defaults name `rally-gpu`; set `RALLY_VM_NAME` or `gcloud delete` directly). The real
gs://+GPU+WASB run caught **4 deploy bugs the droplet/mocks couldn't**, all fixed in **[#318]**
(CI green, merging): (1) `gs://` fetch needs a `dst` (GCS no-passthrough) ? new `_fetch_to`
downloads remote into the workdir (+test, also `_load_gts`); (2) `run_on_vm.sh` must export
`WASB_REPO_DIR`+`WASB_PYTHON` (not just `WASB_WEIGHTS`); (3) weights default
`weights/`?`models/`; (4) manifest `skip_ai_handoff=true` (pure-engine, no paid Gemini ?
deterministic+free). GCP auth = `gcloud` as `avi.dullu@gmail.com`/`gen-lang-client-0306026826`,
GPU quota=1, weights in `models/`, clips+labels+cached-trajectories in the bucket. Suite 1413?.
**? Runner+orchestration now GPU-proven; only the OWNER deploy steps (GCS upload + SMTP secret +
`register_scheduler.sh`) remain to go live.** [[gcp-vm-always-delete]] [[compute-stack-gcp-
only]] Owner: "use GCP GPU VMs (auth/tooling available)." Spun up an **L4** (`rally-nightly-
val`, asia-south1-b) via `create_gpu_vm.sh`, set up the real WASB-native conda env + weights
(`gs://khelsutra-rally-corpus/models/wasb_badminton_best.pth.tar`), ran `nightly_reelcount.py`
with `wasb_hybrid`/`detector_impl=native`/`skip_ai_handoff=true` on
`gs://?/videos/mahadevpura_smoke_180s.mp4`: **22 reels, DETERMINISTIC across 3 runs, clean PASS
(exit 0), faked regression caught (exit 1, `reel_count 25?22`), cost from real uptime
($0.069/?5.77, 352s @ $0.71/hr)**. **VM DELETED** ($0 idle, no orphan disks ? note `teardown.sh`
defaults name `rally-gpu`; set `RALLY_VM_NAME` or `gcloud delete` directly). The real
gs://+GPU+WASB run caught **4 deploy bugs the droplet/mocks couldn't**, all fixed in **[#318]**
(CI green, merging): (1) `gs://` fetch needs a `dst` (GCS no-passthrough) ? new `_fetch_to`
downloads remote into the workdir (+test, also `_load_gts`); (2) `run_on_vm.sh` must export
`WASB_REPO_DIR`+`WASB_PYTHON` (not just `WASB_WEIGHTS`); (3) weights default
`weights/`?`models/`; (4) manifest `skip_ai_handoff=true` (pure-engine, no paid Gemini ?
deterministic+free). GCP auth = `gcloud` as `avi.dullu@gmail.com`/`gen-lang-client-0306026826`,
GPU quota=1, weights in `models/`, clips+labels+cached-trajectories in the bucket. Suite 1413?.
**? Runner+orchestration now GPU-proven; only the OWNER deploy steps (GCS upload + SMTP secret +
```

`register_scheduler.sh`) remain to go live.** [[gcp-vm-always-delete]] [[compute-stack-gcp-only]]
17
18   **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ENGINE REGRESSION ? #315 MERGED + DROPLET-VALIDATED**). master `fa2342c`. Owner asked for a *pure-engine* nightly: run the configured+checkpointed pipeline on 2-3 short clips, assert the **reel count is unchanged** + quality + **cost**, **email** the report, on a **GCP GPU VM** (not local/CI ? GitHub CI can't run WASB). Built `scripts/nightly_reelcount.py` (reel-count=`len(play_segments)` EXACT + R2 quality + `backend.billing` cost USD/INR; exit 0/1/2/3/4; re-run-once guard) + `scripts/nightly_email.py` (SMTP via Secret Manager/env, verified-TLS, graceful skip) + `deploy/gcp/nightly_regression/` (self-deleting `run_on_vm.sh` + driver-agnostic `launch.sh` + `register_scheduler.sh` Cloud Scheduler?Cloud Run job + `create_nightly_bucket.sh`) + `eval_baselines/reelcount_manifest.json` (extensible) + tracked doc `docs/NIGHTLY_ENGINE_REGRESSION.md`. **Complements #202** (cached-trajectory CPU re-score) ? this runs the REAL model E2E. **TWO-BUCKET model (owner-agreed):** ephemeral `gs://khelsutra-rally-nightly` (30-day TTL: reports + repo tarball + build staging) vs permanent `gs://khelsutra-rally-corpus` (clips/labels/weights, no TTL) ? a bucket-wide TTL on the dedicated bucket can't touch golden data. Owner asked for a *pure-engine* nightly: run the configured+checkpointed pipeline on 2-3 short clips, assert the **reel count is unchanged** + quality + **cost**, **email** the report, on a **GCP GPU VM** (not local/CI ? GitHub CI can't run WASB). Built `scripts/nightly_reelcount.py` (reel-count=`len(play_segments)` EXACT + R2 quality + `backend.billing` cost USD/INR; exit 0/1/2/3/4; re-run-once guard) + `scripts/nightly_email.py` (SMTP via Secret Manager/env, verified-TLS, graceful skip) + `deploy/gcp/nightly_regression/` (self-deleting `run_on_vm.sh` + driver-agnostic `launch.sh` + `register_scheduler.sh` Cloud Scheduler?Cloud Run job + `create_nightly_bucket.sh`) + `eval_baselines/reelcount_manifest.json` (extensible) + tracked doc `docs/NIGHTLY_ENGINE_REGRESSION.md`. **Complements #202** (cached-trajectory CPU re-score) ? this runs the REAL model E2E. **TWO-BUCKET model (owner-agreed):** ephemeral `gs://khelsutra-rally-nightly` (30-day TTL: reports + repo tarball + build staging) vs permanent `gs://khelsutra-rally-corpus` (clips/labels/weights, no TTL) ? a bucket-wide TTL on the dedicated bucket can't touch golden data.
19   - **RIGOROUSLY TESTED on the DO droplet** (`claude-do-rally-test` 168.144.126.176, CPU/motion segmenter ? validates the runner+alert+cost, not WASB-on-GPU): ran a couple of times clean (deterministic reels=2, exit 0); **faked TWO issues ? alert caught both** ? behaviour regression (`dead_time_merge_gap` 2?5 ? reels 2?1, exit 1) + a missing clip (pipeline error, exit 1, run completes). **The real-machine run caught 2 bugs the mocked tests couldn't** (now fixed + regression-tested): (1) missing `add_video` ? FK fail ? reels silently 0; (2) one bad clip crashed the whole run (exit 2) since `storage.fetch` passes through missing local paths ? now per-clip isolated. Droplet restored to as-found (`~/rally` removed; `/srv/khelsutra-site` untouched). Adversarially reviewed (45-agent workflow, 41?29 findings, HIGH/MED fixed: SMTP cert-verify, re-run-fail?inconclusive, VM self-delete robustness). Suite 1387?, ruff+mypy+leak-scan clean, all 10 CI green.
20   - ? **OWNER to DEPLOY (the nightly code is merged; going live is owner-gated ? needs GCP secrets/quota I lack):** (1) `bash deploy/gcp/nightly_regression/create_nightly_bucket.sh`; (2) upload 2-3 short owned clips+labels to `gs://khelsutra-rally-corpus/nightly-inputs/` + WASB weights to `weights/`; (3) SMTP app-password ? Secret Manager; (4) `launch.sh` dry-run (the FAITHFUL WASB-on-GPU run + the first `--update-baseline` freeze) ? commit `reelcount_baseline.json`; (5) `register_scheduler.sh`. Two open calls (defaulted, easy to change): **driver** (Cloud Run job vs cron vs Claude routine) + **which clips**. Runbook: `deploy/gcp/nightly_regression/README.md`. [[eval-dual-set-regression]] [[compute-stack-gcp-only]] [[gcp-vm-always-delete]]
21
22   **Checkpoint:** 2026-06-22 (GOLDEN-FIXTURES **P3 ? #317 MERGED**; the golden-fixtures project is now IMPLEMENTATION-COMPLETE). master `7a40434`. Owner cleared ALL owned golden videos (owned+minors-free) + re-confirmed **option (a)** ? minted **6 curated, trimmed (?12k-frame ? few-min), de-attributed REAL Tier-0 fixtures `fx_r01`?`fx_r06`** (single / multi-court×2 / doubles×2 / continuously-active blob; both 29.97/59.94 fps; **~1.9 MB total**) via a NEW `freeze_real_fixture --max-frames` trim. De-attributed (opaque slug, time-origin reset, scenario-only labels, **NO** source name/path/video_id ? grep-verified + leak-scan clean); the characterization + 3-OS cross-OS gates now replay REAL data (F1 0.0?0.59 = real CONTROL-path, **behaviour-locking NOT field quality**; hard multi-court/blob collapse to a mega-window F1=0 = the known over-merge, now frozen). DATA found on this box: `C:\Users\avidu\Projects\Annotation Setup\Collect\{Trajectories,Golden Labelled}\` + repo `output\*.rallies.csv` (manifest `output/human_lovo_manifest.json` = 15 videos); F: has the source GoPro videos (not needed). **Surrogate?source map kept OFF-git** (given to owner in chat: fx_r01=mahadevpura_single,

r02=kushagra, r03=mahadevpura_1, r04=largetest_doubles, r05=GX010128, r06=Boxhill_Doubles). Suite **1390?/7s**, cov **85.27%**, all CI green incl. leak-scan + 3-OS. **Golden-fixtures tracker now M1/M2/M3/MX/P1/P2/DOC/P0/P3 ALL ?** (P0=safe-scope; the FUNCTIONAL video-name rename DEFERRED to a coordinated pre-open-source effort; M4 quality-floor + O1 protobuf + O2 Tier-1-key-gated remain OPTIONAL). Track PRs = #186/#189/#191/#192/#193/#194 + #314 + #317, all merged+audited. [[golden-set-vendoring]] [[gotcha-shared-checkout-branch-collisions]]
23
24   **Checkpoint:** 2026-06-22 (**khelsutra brand-repo HYGIENE + DOC-FRESHNESS SWEEP ? 2 PRs merged, clean slate**). Session = "sync to remote head + low-hanging fruit + quick doc sweep." khelsutra was at master `e2570e4` (#93). ? **[#94] hygiene** ? `.gitignore` now covers `*.db` (runtime `site/access.db`) + `.claude/` (machine-specific; both were untracked ? committable) + corrected a stale NEXT_STEPS PR count; **caught my own `.gitignore` inline-comment bug via `git check-ignore` BEFORE commit** (`.gitignore` has no trailing-comment syntax ? comments must be own-line). ? **[#95] doc-freshness sweep** ? a **15-agent AUDIT** (one verifier/doc, every falsifiable claim checked at file:line incl. cross-repo vs engine `origin/master`) found **44 stale findings** (the pattern: docs froze at their authoring-PR while the SPA / engine API / localization / alpha-deploy shipped past them) ? a **12-agent FIXER fan-out** applied **63 surgical edits across 13 docs**; I **independently re-verified every load-bearing engine claim** on engine `origin/master` before merging (`/api/{health:308,capabilities:340,videos:374}` + compile `download_url:768` + `native_wasb_runner.py` on master + `auth.py::CF_ACCESS_HEADER` + `_allowed_hosts_from_env` REPLACE-semantics ? all real; brittle engine line-anchors ? symbol refs like `main.py::compile_highlights`). Honesty held (P6 stays `?` engine-done/SPA-pending, native-WASB "not production-verified", footage shots owner-gated, historical changelogs preserved; the 2 `fresh` docs untouched). Also pruned **6 stale merged local branches**. **All CI green (site+web+layout); 0 open PRs; only `master` locally; khelsutra master now `7dc7de5`.** *(Brand/site track ? distinct from the serving/packaging/UX/golden threads below.)* [[lesson-verify-engine-surface-before-scoping]] [[working-agreement-merging]] [[doc-status-register]]
25
26   **Checkpoint:** 2026-06-22 (GOLDEN-FIXTURES **P0 ? #314 MERGED**, master `5c2eec8`; a SEPARATE thread from packaging/UX). Owner: "all owned golden videos are owned+minor-free; pick (a); make P0 now." On scouting, P0 was bigger/riskier than the audit's "4 files": the video names are FUNCTIONAL (eval_baselines keys ? training floor + the gitignored `output/human_lovo_manifest.json` + data files; `MULTICOURT_CLIPS`; hot in QUALITY_ITERATIONS/the quality track) ? surfaced it ? owner picked **SAFE SCOPE**. Shipped: scrubbed personal `C:\Users\avidu` paths from 3 code docstrings + **`tools/leak_scan.py`** (structural PII guard: 32-hex MD5 + personal paths over backend/training/scripts/tools/eval_baselines; tests/ excluded; optional gitignored `.leakscan_private` for local name scanning) + CI `leak-scan` job + 10 tests. Clean scan; suite **1365?/7s**, cov **85.27%**, all CI green. **DEFERRED (safe-scope):** the functional video-name rename + git-history rewrite (need manifest/data/training-floor coordination + quality-track pause; do before open-sourcing). ?? **NEXT = P3 real Tier-0 fixtures** ? owner APPROVED (all owned cleared + **(a)** commit de-attributed per-frame CSV); needs the golden **trajectory CSVs + label CSVs** (`Annotation Setup/Collect/Trajectories/` + `Golden Labelled/` / OneDrive `golden-data/`) ? I locate them or owner points me, then `--freeze-real` per video. Golden-fixtures track = #186/#189/#191/#192/#193/#194 + #314, all merged+audited. [[golden-set-vendoring]]
27
28   **Checkpoint:** 2026-06-22 (? **GCS = corpus source-of-truth ? `docs/DATA_IN_GCS.md` + Phase-1 metadata MIGRATED ? PR [#316](https://github.com/Khelsutra/badminton-highlight-indexer/pull/316) OPEN; F: label backup done**). Owner goal: run ALL train+infer from the cloud with ALL data in `gs://khelsutra-rally-corpus` ? remove the local box as a blocker; document where data lives so other sessions pick up from the bucket. [[gcs-data-source-of-truth]]
29   - ? **Authoritative map `docs/DATA_IN_GCS.md`** (indexed in docs/README.md; branch `work/golden-corpus-hygiene`, commit `8884c9f`): bucket layout · current contents + **drift** · how `backend.worker infer|train` consume `gs://` · the gap · phased migration plan + DoD. The **cloud engine already exists** (archived DECOUPLED_COMPUTE: `worker infer --video-uri` / `worker train --corpus-uri`, URI backends in `backend/storage/ref.py`, `deploy/gcp/`) ? the doc records WHERE bytes are, it does NOT redesign.
30   - ? **Gap measured:** corpus *metadata* is **~25 MB & entirely local** ? 20 trajectory CSVs (17.5 MB at `C:\?\Collect\Trajectories\`), `output/human_lovo_manifest.json` (C:-absolute paths), most labels. Bucket has only 3 videos, **no `trajectories/`**, inconsistent `_proxy` label naming. Only the VIDEOS are large ? migrate **Drive?bucket from a cloud box**, not via the local box.

164. assistant (2026-06-22T12:49:32.519Z)

Let me prepend a concise checkpoint to current-state and add the memory index pointer:

165. user (2026-06-22T12:49:45.013Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

166. assistant (2026-06-22T12:49:50.630Z)

Now add the index pointer in MEMORY.md:

167. user (2026-06-22T12:49:55.332Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

168. user (2026-06-22T12:49:59.106Z)

55   - [Sibling internship repo](sibling-internship-repo.md) ? khelacharya shot-intelligence layer builds ON TOP of our rally segmentation; kept off master at branch level (NOT a separate GitHub repo ? corrected 2026-06-06)
56   - [Paper co-author aim](paper-coauthor-aim.md) ? **standing side-aim**: co-author a paper on "local, cheap & efficient rally detection via multiple clever ideas". Keep `docs/QUALITY_ITERATIONS.md` as the ablation-ready empirical backbone; honest measured results ONLY.
57

169. user (2026-06-22T12:50:02.698Z)

42:- [Eval dual-set regression](eval-dual-set-regression.md) ? owner directive (2026-06-16, proven practice): keep a **RAW eval set** (currently-bad videos, random sample) + a **GROWING GOLDEN-finalized eval set** (promote raw?golden when at-par) + a separate **golden TRAINING set**; every quality launch must NOT regress on EITHER eval set (don't lose a graduated win). The **ablation layer A/B**s these. Checkpointed video folder; wire dual-set aggregate into `run_regression.py` when R2 lands.

170. user (2026-06-22T12:50:05.699Z)

42   - [Eval dual-set regression](eval-dual-set-regression.md) ? owner directive (2026-06-16, proven practice): keep a **RAW eval set** (currently-bad videos, random sample) + a **GROWING GOLDEN-finalized eval set** (promote raw?golden when at-par) + a separate **golden TRAINING set**; every quality launch must NOT regress on EITHER eval set (don't lose a graduated win). The **ablation layer A/B**s these. Checkpointed video folder; wire dual-set aggregate into `run_regression.py` when R2 lands.

171. user (2026-06-22T12:50:13.243Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

172. assistant (2026-06-22T12:50:17.856Z)

Let me confirm PR #324's CI (the litmus skips in CI, so it should pass cleanly):

173. user (2026-06-22T12:50:24.609Z)

checks ok=0/0 pending/other=0
=== final ===
{"failures":[],"mergeStateStatus":"DIRTY","mergeable":"CONFLICTING"}

174. assistant (2026-06-22T12:50:32.090Z)

`#324` shows `CONFLICTING` ? unexpected, since I branched from `db74f9b` and only touched 2 files. Master may have moved again (sibling merge). Let me check:

175. user (2026-06-22T12:50:36.885Z)

```
=== origin/master HEAD now ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden
corpus (#322)
db74f9b docs(data): mark DATA_IN_GCS §7b blocker #1 resolved by #319 (#321)
=== my branch base ===
5edd9f8 test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus
=== did master move past db74f9b? merge-base ===
master advanced beyond my base
=== files changed on master since my base that overlap mine ===
docs/QUALITY_ITERATIONS.md
tests/test_served_gate_a_regression.py
=== what changed on master since db74f9b ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden
corpus (#322)
```

176. assistant (2026-06-22T12:50:50.695Z)

Key finding: PR **#322** (`32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden corpus`) **already landed the same fix** ? a parallel session/owner fixed master while I was working (the shared-checkout reality again). My #324 is now a **duplicate** and conflicts because it touches the same two files. Let me verify #322 actually fixes it correctly before closing mine:

177. user (2026-06-22T12:50:54.503Z)

```
=== #322's pinned values in the litmus (master) ===
18:SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has since
28:same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation (1.36?0.79).
108:    # Honest measured value (2026-06-22, present 15-clip multicourt golden set): mean served
109:    # ?F1 ? -0.023 (was ? -0.034 on the 9-clip / ? -0.008 on the retired original-6). Band
admits
111:    assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
113:    assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
178:    # Honest measured snapshot (2026-06-22, present 15-clip multicourt set): mean ?F1 ?
+0.03,
179:    # over-seg 1.36 ? 0.79 (was -0.001 / 1.16 ? 0.64 on the 9-clip; +0.074 / 1.68 ? 1.01 on
the
181:    assert mean_delta == pytest.approx(0.03, abs=0.03), mean_delta
182:    assert mean_seg_off == pytest.approx(1.36, abs=0.10), mean_seg_off
183:    assert mean_seg_on == pytest.approx(0.79, abs=0.10), mean_seg_on

=== does master's litmus pass on the live corpus? ===
(saved master copy)
```

178. assistant (2026-06-22T12:51:19.102Z)

#322's pins are **byte-identical to mine** (`approx(0.03)`, `approx(1.36)`, `approx(0.79)`, same bands) ? two independent re-measurements reached the same numbers, which strongly cross-validates the re-grounding. Master is already fixed by #322. Let me definitively confirm master's litmus passes on the live corpus, then clean up my redundant #324.

179. user (2026-06-22T12:51:41.603Z)

```
=== run MASTER's exact litmus file against the live corpus ===
.....                                                                    [100%]
```

```
5 passed in 19.76s
=== restore my branch's copy ===
restored
```

180. assistant (2026-06-22T12:51:59.058Z)

Confirmed: master (with #322) passes the litmus ? 5 passed. Master is already fixed; my #324 is a redundant duplicate that independently reached the identical pinned values. Let me close it cleanly and delete its branch.

181. user (2026-06-22T12:52:09.304Z)

```
? Closed pull request Khelsutra/badminton-highlight-indexer#324 (test(eval): re-ground served
Gate-A litmus to the 15-clip golden corpus)
=== delete remote branch ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 - [deleted]         fix/served-gate-a-regrounded-15clip
=== switch off the dead branch + delete locally ===
error: pathspec 'fix/worker-train-dated-label-join' did not match any file(s) known to git
error: cannot delete branch 'fix/served-gate-a-regrounded-15clip' used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer'
```

182. assistant (2026-06-22T12:52:27.112Z)

#324 closed and its remote branch deleted. The local `fix/worker-train-dated-label-join` was already pruned by a sibling, so let me park on a detached `origin/master` (master itself is locked by another worktree) and drop the redundant local branch:

183. user (2026-06-22T12:52:29.042Z)

```
HEAD is now at 9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0
rename) + optional O1/O2 (#323)
Deleted branch fix/served-gate-a-regrounded-15clip (was 5edd9f8).
=== where am I ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
```

HEAD (no branch)

184. assistant (2026-06-22T12:52:36.627Z)

Now let me correct my memory/checkpoint notes ? the actual merged fix is **#322** (parallel session), not my closed-duplicate #324:

185. user (2026-06-22T12:52:45.137Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\golden-corpus-grew-9-to-15.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

186. user (2026-06-22T12:52:53.453Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ?
no need to Read it back)
```

187. user (2026-06-22T12:53:03.256Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

188. assistant (2026-06-22T12:53:30.226Z)

Everything is reconciled. Here's the complete picture.

? Both requested PRs merged

- **[#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)** ? `worker train` dated-label join fix (`_label_clip_name` helper + 6 tests, worker coverage 82%?84%). Squash-merged ? `b72e6de`.
- **[#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321)** ? the doc fix: `DATA_IN_GCS.md` §7b blocker #1 flipped to ? RESOLVED + §8 tracker row. Squash-merged ? `db74f9b`.

Both passed a 3-dimension adversarial verification workflow (code-correctness, doc-accuracy/no-fabrication, test-rigor) ? `all_ok: true`, zero blocking issues ? before I merged them.

? Why master was "broken" ? and it's already fixed

Diagnosis: not a code regression. Master's *CI* is green; only the *local* full suite failed, on one test:

```
test_served_gate_a_regression::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg
  assert mean_seg_off == 1.16 ± 0.10  ->  got 1.36
```

That litmus pins **date-stamped over-segmentation snapshots** measured on a **9-clip** golden set, and **skips in CI** (its manifest is gitignored) ? so it can only break locally. The **#316 GCS migration restored the original-6 single-court labels**, growing the local corpus **9 ? 15 clips**. Single-court clips over-segment, so the blend moved `mean_seg_off` to 1.36. The litmus tripped **exactly as designed** ("snapshots will need re-measurement as the corpus grows"). I confirmed every *durable* invariant still holds ? corpus change, not a bug.

The fix was already merged in parallel. While I was re-measuring and re-grounding it, another session landed the **identical** fix as **[#322](https://github.com/Khelsutra/badminton-highlight-indexer/pull/322)** (`32f4ee6`). My independent re-measurement reached the **same** pins (`mean_seg_off` 1.36, `mean_seg_on` 0.79, proposal ?F1 ? +0.03, served ?F1 ? ?0.023) ? a nice cross-validation. So I **closed my duplicate [#324]** and deleted its branch. I verified `origin/master` (now `9b37269`, includes #322) passes the litmus locally: **`5 passed`**.

Master is green ? both broken thing (the litmus) and the worker-train bug are resolved.

Notes

- A genuine finding surfaced: with the original-6 back (they carried the archived **+0.074** Gate-A win), the proposal-windowing gate is now mildly **F1-positive** again, while the served operating point stays mildly negative. The corpus 9?15 growth likely affects other eval narratives ? worth a wider sweep sometime.
- I left a re-usable measurement helper at `scratch/measure_served_gate_a_litmus` (gitignored) for re-grounding this litmus next time the corpus changes.
- Recorded the corpus-state change as a memory so a future session hitting this litmus failure checks corpus size before suspecting a regression.

One housekeeping note from the shared checkout: this repo has `master` checked out in another worktree (`tier1-windowing`) and several sibling sessions active ? that's how #322 landed underneath me. I branched everything from `origin/master` and staged explicit paths throughout, so nothing collided.

189. user (2026-06-22T13:22:34.377Z)

so no pending items for you except the wider sweep which can be done after more iterations?

190. assistant (2026-06-22T13:23:01.788Z)

Good question ? let me not just agree; let me verify it on the actual current master before confirming. I'll run the full suite on `9b37269` (where I'm parked) and scan for any *other* eval tests/docs still pinned to the old 9-clip corpus.

191. user (2026-06-22T13:23:02.768Z)

```
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
=== launch full suite on current master ===
```

192. user (2026-06-22T13:23:06.057Z)

```
Command running in background with ID: bq01qap6h. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bq01qap6h.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

193. assistant (2026-06-22T13:23:09.008Z)

```
Now scan for any *other* eval tests/docs still pinned to the old 9-clip corpus (the scope of the
"wider sweep"):
```

194. user (2026-06-22T13:23:10.314Z)

```
Found 8 files
docs\QUALITY_ITERATIONS.md
tests\test_served_gate_a_regression.py
docs\ABLATION_LAYER.md
tests\test_ablation.py
docs\archives\quality-iterations\README.md
tests\test_per_user_gate.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
```

195. assistant (2026-06-22T13:23:21.090Z)

```
Let me characterize those hits ? are `test_ablation.py` / `test_per_user_gate.py` corpus-pinned
(would break on 15 clips), and is `ABLATION_LAYER.md` carrying the stale 9-clip narrative?
```

196. user (2026-06-22T13:23:24.994Z)

```
=== test_ablation.py: skip-gating + any 9/1.16 pins ===
13:  - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
15:    cleanly when the golden JSONs are absent (CI), runs when present (owner box).
205:    assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
221:    assert abs(row.delta_f1) < 1e-9                  # honestly a wash
252:        assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
324:    pytest.importorskip("reportlab")
325:    pypdf = pytest.importorskip("pypdf")
332:    assert os.path.isfile(out)
353:    with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
361:    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
369:    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
390:# ------------------------------------------------------------------------ the golden
litmus
396:def _golden_present() -> bool:
397:    """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
402:    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
413:@pytest.mark.skipif(not _golden_present(),
414:                     reason="golden output/*_golden_features.json absent (run fusion_golden
first)")
416:    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured
Gate-A
419:    where the golden substrate is absent; runs on the owner box where it is present."""
428:    assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
430:    assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
431:    assert row.ci_low == pytest.approx(0.042, abs=0.01)
432:    assert row.ci_high == pytest.approx(0.104, abs=0.01)
```

```
436:    # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
437:    assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)

=== test_per_user_gate.py: the matched lines ===
67:    a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
83:    a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]

=== ABLATION_LAYER.md: the matched lines (context) ===
73:| over-seg ratio | 1.68 ? 1.01 | 1.68 ? 1.01 |
```

197. assistant (2026-06-22T13:23:49.643Z)

Let me wait for the full-suite result on master (the authoritative check), then answer precisely:

198. user (2026-06-22T13:24:50.063Z)

```
.................................................................... [ 95%]
.........................................................            [100%]
1430 passed in 98.85s (0:01:38)
============================================================================
 RESULT: PASS - all tests green
============================================================================
```

199. assistant (2026-06-22T13:25:20.776Z)

Correct ? confirmed with evidence. Full suite on current master (`9b37269`) is **all green: 1430 passed**, and CI is green too.

Nothing pending for me from this work:
- [#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319) (worker-train fix) + [#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321) (doc) ? merged.
- The local litmus break ? fixed by [#322](https://github.com/Khelsutra/badminton-highlight-indexer/pull/322) (parallel session); my duplicate #324 closed; no open PRs or uncommitted work left by me.

One honest correction on the "wider sweep" ? I overstated it last turn. Having now scanned the other eval tests/docs, it's narrower than "likely affects other narratives," and may be a non-issue:
- I checked the other corpus-pinned litmus, `tests/test_ablation.py` (+ `docs/ABLATION_LAYER.md`). It pins the original-6 **+0.074 / over-seg 1.68?1.01** on a *separate* substrate (`output/*_golden_features.json`), **not** the `human_lovo_manifest.json` that grew to 15 ? so the 9?15 change doesn't touch it, and it's green.
- The other grep hits were either historical/archived records (correct as-is) or false positives (`test_per_user_gate.py`'s `0.64` is synthetic test data, not a corpus number).

So there's **no known-stale eval item remaining** ? the served litmus was the only thing the corpus growth broke, and it's fixed. The "wider sweep" is now just an *optional, low-priority* sanity pass (e.g., if the owner later wants to re-derive any original-6-vs-blend comparisons now that the original-6 are back in the corpus). It can absolutely wait for more iterations, and it may well turn out to be a no-op.

For completeness, the genuinely-open items in this area are **not mine and pre-date this task**: `DATA_IN_GCS.md` §7b blockers #2?#5 (Phase-3 eval-on-GCS plumbing, `manifests/golden.json` having no consumer, GPU-quota/secrets gating, promoting the `scratch/` migration helpers) and the nightly-regression owner deploy steps ? all owner-gated or separately tracked.